

Cryptographic Hash Algorithm uHash

Yafei Zheng, Yuhan Zhang, Han Sui,
Meichun Jia, Si Gao, Wenling Wu

¹ Institute of Software Chinese Academy of Sciences, Beijing, P.R.China

² Zhongguancun Laboratory, Beijing, P.R.China

³ University of Chinese Academy of Sciences, Beijing, P.R.China

Abstract. This report first elaborates on the cryptographic hash algorithm uHash, followed by a brief introduction to the design principles, preliminary security analysis and evaluation, software and hardware implementation. uHash is a cryptographic hash algorithm constructed based on block cipher (specifically uBlock-1024). uHash supports digest lengths of 512, 768 and 1024 bits, denoted as uHash-512, uHash-768 and uHash-1024 respectively. With reasonable cryptographic assumptions, uHash is equipped with security proofs and analysis for the overall architecture. It can resist known attacks and quantum computing attacks. Specifically, the classical security against collision attacks is no less than $h/2$ bits, and the classical security against preimage and second-preimage attacks is no less than h bits. Meanwhile, security against quantum computing attacks is not lower than generic quantum attacks. uHash can be efficiently implemented on a wide range of software and hardware platforms.

Keywords: Hash algorithm · Block cipher · Cryptanalysis · Implementation

Contents

1	Description of uHash	3
1.1	Notations	3
1.2	Algorithm Specification	3
2	Design Principle of uHash	9
2.1	Parameters	9
2.2	Message Padding	9
2.3	Overall Architecture	9
2.4	Underlying Block Cipher	10
3	Security Evaluation of uHash	11
3.1	Security Claims	11
3.2	Security Proof	11
3.3	Security Supplement	22
4	Security of uBlock-1024	23
4.1	Differential Cryptanalysis	23
4.2	Linear Cryptanalysis	23
4.3	Impossible Differential Cryptanalysis	24
4.4	Integral Cryptanalysis	24
4.5	Meet-in-the-Middle Attack	24
4.6	Boomerang Attack	25
4.7	Differential-linear Attack	27
5	Performance of uHash	27
5.1	Software Implementation	28
5.2	Hardware Implementation	28
5.3	Other Platforms	29
6	Features of uHash	30
7	Statistical Tests of uHash	31
7.1	Statistical Test of uHash-512	31
7.2	Statistical Test of uHash-768	31
7.3	Statistical Test of uHash-1024	31
8	Test Vectors of uHash	31
8.1	Test Vectors of uHash-512	33
8.2	Test Vectors of uHash-768	34
8.3	Test Vectors of uHash-1024	36

Introduction

In response to the Next-generation Commercial Cryptographic Algorithm Program (NGCC) launched by the Institute of Commercial Cryptography Standards, we have developed a cryptographic hash algorithm named uHash. This report presents the algorithm specification, design philosophy, preliminary security analysis, and performance of implementation.

The design objective of uHash is to construct a high-security cryptographic hash algorithm based on block ciphers. The main technical specifications comply with the submission requirements of NGCC, which are detailed as follows:

- The submitted algorithm shall support message digest lengths of 512 and 1024 bits at the same time, and optionally support message digest length of 768 bits. The submitted algorithm shall support a maximum message length of at least $2^{64} - 1$ bits.
- The submitted algorithm shall resist all known classical and quantum attacks.
- The submitted algorithms shall be implemented efficiently on a wide range of software and hardware platforms.

1 Description of uHash

1.1 Notations

The notations used in the report are defined as Table 1.

Table 1: Notations

h	message digest length, $h = 512/768/1024$
n	block size of the underlying block cipher, $n = 256$
b	number of branches of the compression function, $b = 2/3/4$
bn	size of the hash algorithm/compression function, $bn = h$
L	bit length of the message
l	number of message chunks
a	number of blocks in a message chunk, $a = 3/2/1$
IV	initial value
u	bn -bit chaining value
u_i^j	the j -th branch of the i -th chaining value
$\langle X \rangle_c$	c -bit binary representation of X
$E_K(X)$, $E(X, K)$	encrypt X with block cipher E under key K (For uHash, E is uBlock-1024)
$\oplus$	modulo-2 addition
$\lll_{32} c$	32-bit word-wise cyclic left shift by c bits
$\parallel$	concatenation of bit strings

1.2 Algorithm Specification

The cryptographic hash algorithm uHash supports message digest lengths of 512, 768, and 1024 bits, and the corresponding algorithms are denoted as uHash- h or uHash- bn ($h = bn = 512/768/1024$). Given any message M , the algorithm outputs an h -bit digest H .

The uHash algorithm is described as Algorithm 1, where M_i ($i = 1, 2, \dots, l$) is a message chunk of an bits, satisfying $a = 5 - b$.

Algorithm 1 The uHash- h Algorithm

```

1: uHash- $h$ ( $IV, M$ )
2:  $M' \leftarrow \text{padding}(M)$  ▷ Message padding
3:  $M_1 \parallel M_2 \parallel \dots \parallel M_l \leftarrow M'$  ▷ Message splitting
4:  $u_1 \leftarrow IV$  ▷ Initialization
5: for  $i = 1$  to  $l - 1$  do ▷ Iteration
6:    $u_i^1 \parallel u_i^2 \parallel \dots \parallel u_i^b \leftarrow u_i$ 
7:    $K_i \leftarrow u_i^2 \parallel \dots \parallel u_i^b \parallel M_i$ 
8:   for  $j = 1$  to  $b$  do
9:      $u_{i+1}^j \leftarrow E_{K_i}(u_i^1 \oplus (j - 1)) \oplus u_i^1$ 
10:  end for
11: end for
12:  $K_l \leftarrow u_l^2 \parallel \dots \parallel u_l^b \parallel M_l$ 
13: for  $j = 1$  to  $b$  do ▷ Final iteration
14:    $u_{l+1}^j \leftarrow E_{K_l \oplus (j+15)}(u_l^1 \oplus (j - 1)) \oplus u_l^1$ 
15: end for
16:  $H \leftarrow u_{l+1}^1 \parallel u_{l+1}^2 \parallel \dots \parallel u_{l+1}^b$  ▷ Digest output
17: return  $H$ 

```

Table 2: Parameters of uHash- h

Algorithm	Digest length h	Branches b	Blocks/Chunk a
uHash-512	512	2	3
uHash-768	768	3	2
uHash-1024	1024	4	1

1.2.1 Message Padding

Given message M , append a single bit ‘1’ followed by the minimum number of ‘0’ bits (possibly zero), such that the length of the padded output $M' = M \parallel 1 \parallel 0 \dots 0$ is a positive integer multiple of an bits.

1.2.2 Message Splitting

Split the padded string M' into l message chunks of an bits each, i.e. $M' = M_1 \parallel M_2 \parallel \dots \parallel M_l$.

1.2.3 Initialization

The bn -bit initial value IV is assigned to the initial state u_1 . $IV = IV_1 \parallel \dots \parallel IV_b$, and each IV_j is defined as:

$$\begin{aligned}
IV_1 &= 0^{128} \parallel 0^{64} \parallel \langle bn \rangle_{64} \\
IV_2 &= 0^{128} \parallel \langle bn \rangle_{64} \parallel 0^{64} \\
IV_3 &= 0^{64} \parallel \langle bn \rangle_{64} \parallel 0^{128} \\
IV_4 &= \langle bn \rangle_{64} \parallel 0^{64} \parallel 0^{128}
\end{aligned}$$

Table 3: Initial values

Algorithm	IV
uHash-512	$IV_1 \parallel IV_2$
uHash-768	$IV_1 \parallel IV_2 \parallel IV_3$
uHash-1024	$IV_1 \parallel IV_2 \parallel IV_3 \parallel IV_4$

1.2.4 Compression Function

The compression function CF_{bn} is iteratively called to process the first $l - 1$ message chunks. $CF_{bn} : \{0, 1\}^{bn} \times \{0, 1\}^{an} \rightarrow \{0, 1\}^{bn}$ is defined as:

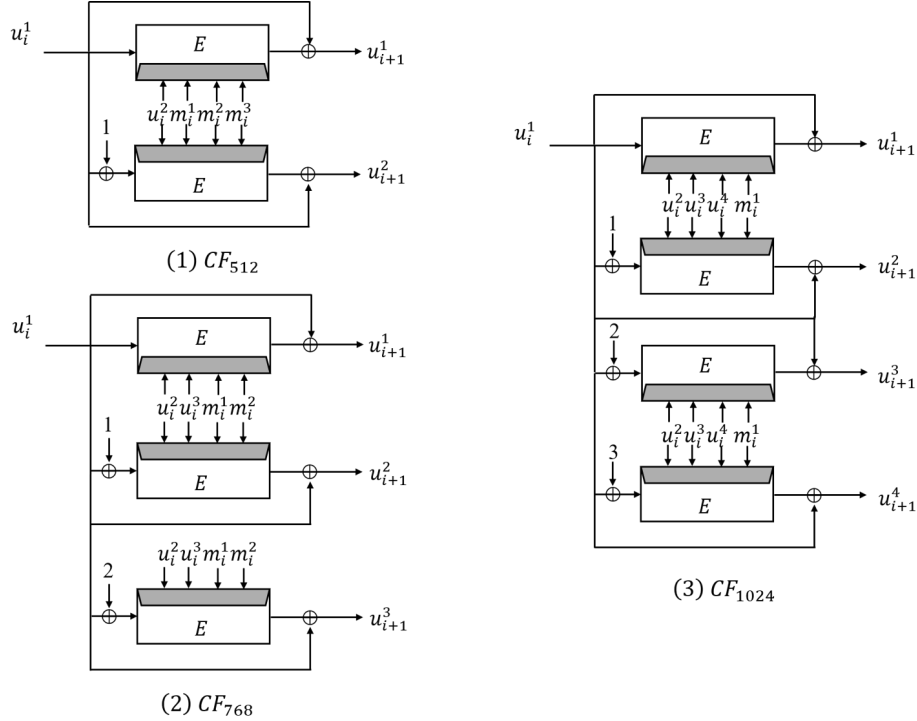
$$CF_{bn}(u_i^1 \parallel u_i^2 \parallel \dots \parallel u_i^b, M_i) = (u_{i+1}^1 \parallel u_{i+1}^2 \parallel \dots \parallel u_{i+1}^b),$$

where for each $j \in \{1, 2, \dots, b\}$:

$$u_{i+1}^j = E_{K_i}(u_i^1 \oplus (j-1)) \oplus u_i^j.$$

E denotes block cipher uBlock-1024, which supports n -bit block length and $4n$ -bit key length, and

$$K_i = u_i^2 \parallel \dots \parallel u_i^b \parallel M_i.$$

**Figure 1:** Compression functions of uHash

1.2.5 Final Iteration

When processing the final message chunk, CF'_{bn} is used in place of CF_{bn} . The only difference between the two is that the key in CF'_{bn} is XORed with additional counters, i.e.,

$$u_{l+1}^j = E_{K_l \oplus (j+15)}(u_l^1 \oplus (j-1)) \oplus u_l^1.$$

If there is only one message chunk M_1 after message padding, CF'_{bn} is directly called to process M_1 .

1.2.6 Message Digest Output

After processing all l message chunks, the state u_{l+1} is output as the digest H .

1.2.7 Underlying Block Cipher

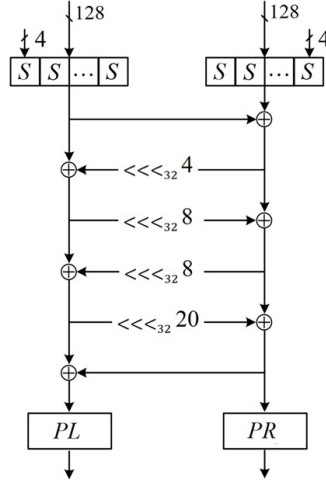
The underlying block cipher uBlock-1024 is a variant of block cipher uBlock [1] with 256-bit block and 1024-bit key. The encryption algorithm consists of 52 rounds of iterative transformations, and the round function is shown as Figure 2. It takes a 256-bit plaintext X and round keys $RK^0, RK^1, \dots, RK^{51}$ as input, and outputs a 256-bit ciphertext Y . The encryption algorithm is described as:

$$\begin{aligned} & \text{uBlock-1024}(X, RK^0, RK^1, \dots, RK^{51}) \\ & X_0 \parallel X_1 \leftarrow X \\ & \text{for } i = 0 \text{ to } 51 \text{ do} \\ & \quad RK_0^i \parallel RK_1^i \leftarrow RK^i \\ & \quad X_0 \leftarrow S_{32}(X_0 \oplus RK_0^i) \\ & \quad X_1 \leftarrow S_{32}(X_1 \oplus RK_1^i) \\ & \quad X_1 \leftarrow X_1 \oplus X_0 \\ & \quad X_0 \leftarrow X_0 \oplus (X_1 \lll_{32} 4) \\ & \quad X_1 \leftarrow X_1 \oplus (X_0 \lll_{32} 8) \\ & \quad X_0 \leftarrow X_0 \oplus (X_1 \lll_{32} 8) \\ & \quad X_1 \leftarrow X_1 \oplus (X_0 \lll_{32} 20) \\ & \quad X_0 \leftarrow X_0 \oplus X_1 \\ & \quad X_0 \leftarrow PL(X_0) \\ & \quad X_1 \leftarrow PR(X_1) \\ & \text{end for} \\ & Y \leftarrow X_0 \parallel X_1 \end{aligned}$$

S_{32} is constructed by arranging in parallel 32 identical 4-bit S-boxes.

$$\begin{aligned} S_{32} : (\{0, 1\}^4)^{32} &\rightarrow (\{0, 1\}^4)^{32} \\ (x_0, x_1, \dots, x_{31}) &\mapsto (s(x_0), s(x_1), \dots, s(x_{31})) \end{aligned}$$

The 4-bit S-box s is given by Table 4.

**Figure 2:** Round function of uBlock-1024**Table 4:** S-box

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$s(x)$	3	0	6	2	5	4	f	e	a	8	7	9	1	c	d	b

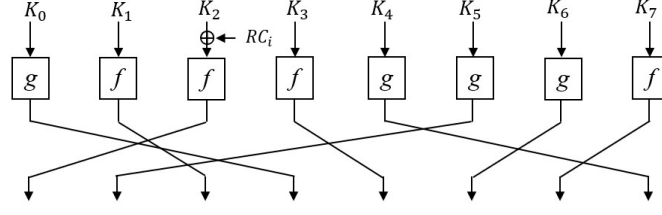
PL and PR are permutations of 32 nibbles.

$$\begin{aligned}
 PL: & (\{0, 1\}^4)^{32} \rightarrow (\{0, 1\}^4)^{32} \\
 & (y_0, y_1, \dots, y_{31}) \mapsto (z_0, z_1, \dots, z_{31}) \\
 & z_0 = y_{28}, z_1 = y_{18}, \dots, z_{31} = y_8 \\
 PR: & (\{0, 1\}^4)^{32} \rightarrow (\{0, 1\}^4)^{32} \\
 & (y_0, y_1, \dots, y_{31}) \mapsto (z_0, z_1, \dots, z_{31}) \\
 & z_0 = y_0, z_1 = y_2, \dots, z_{31} = y_{21}
 \end{aligned}$$

Table 5: Permutations PL and PR

PL	{28, 18, 16, 26, 30, 22, 29, 23, 27, 17, 24, 31, 25, 19, 21, 20, 3, 10, 4, 15, 14, 5, 7, 13, 12, 0, 11, 1, 9, 2, 6, 8}
PR	{0, 2, 1, 12, 11, 8, 13, 7, 10, 4, 6, 5, 3, 14, 9, 15, 22, 18, 27, 23, 24, 31, 20, 28, 29, 17, 16, 19, 25, 30, 26, 21}

The 1024-bit master key K is split into eight 128-bit words and used as subkeys RK^0, RK^1, RK^2, RK^3 . The master key is then updated every 4 rounds, and be taken as subkeys $RK^{4i}, RK^{4i+1}, RK^{4i+2}, RK^{4i+3}$, $i = 1, 2, \dots, 12$.

**Figure 3:** Update function of key

 $KeySchedule(K, RC_1, RC_2, \dots, RC_{12})$

 $K_0 \| K_1 \| K_2 \| K_3 \| K_4 \| K_5 \| K_6 \| K_7 \leftarrow K$
 $RK^0 \leftarrow K_0 \| K_1$
 $RK^1 \leftarrow K_2 \| K_3$
 $RK^2 \leftarrow K_4 \| K_5$
 $RK^3 \leftarrow K_6 \| K_7$

 for $i = 1$ to 12 do

 $K \leftarrow f(K_2 \oplus RC_i) \| g(K_5) \| f(K_1) \| g(K_0) \| f(K_3) \| g(K_6) \| f(K_7) \| g(K_4)$
 $K_0 \| K_1 \| \dots \| K_7 \leftarrow K$
 $RK^{4i} \leftarrow K_0 \| K_1$
 $RK^{4i+1} \leftarrow K_2 \| K_3$
 $RK^{4i+2} \leftarrow K_4 \| K_5$
 $RK^{4i+3} \leftarrow K_6 \| K_7$

 end for

f and g are permutations of 32 nibbles.

Table 6: Permutations f and g

f	{11, 3, 14, 2, 9, 10, 12, 0, 4, 13, 7, 8, 15, 6, 5, 1, 16, 19, 27, 22, 31, 26, 20, 30, 23, 29, 25, 18, 24, 21, 28, 17}
g	{23, 3, 31, 2, 0, 20, 21, 15, 13, 29, 14, 30, 12, 28, 22, 1, 17, 24, 8, 5, 27, 16, 6, 11, 25, 18, 19, 10, 7, 9, 26, 4}

The 32-bit round constant RC_i is generated by an 8-stage LFSR with initial conditions $c_0 = c_3 = c_6 = c_7 = 0$, $c_1 = c_2 = c_4 = c_5 = 1$. For $i \geq 8$,

$$c_i = c_{i-2} \oplus c_{i-3} \oplus c_{i-7} \oplus c_{i-8}.$$

Let

$$a_i = c_i \bar{c}_{i+1} c_{i+2} c_{i+3} c_{i+4} c_{i+5} c_{i+6} c_{i+7},$$

$$a'_i = c_i \bar{c}_{i+1} c_{i+2} \bar{c}_{i+3} c_{i+4} \bar{c}_{i+5} c_{i+6} c_{i+7},$$

$$a''_i = c_i c_{i+1} c_{i+2} \bar{c}_{i+3} c_{i+4} c_{i+5} c_{i+6} \bar{c}_{i+7},$$

$$a'''_i = c_i c_{i+1} c_{i+2} c_{i+3} c_{i+4} \bar{c}_{i+5} c_{i+6} \bar{c}_{i+7},$$

then

$$RC_i = a_i \| a'_i \| a''_i \| a'''_i.$$

The hexadecimal representations of the constants RC_i ($i = 1, 2, \dots, 12$) are given in Table 7.

Table 7: Round constants RC_i

RC_1	988CC9DD	RC_2	F0E4A1B5
RC_3	21357064	RC_4	8397D2C6
RC_5	C7D39682	RC_6	4F5B1E0A
RC_7	5E4A0F1B	RC_8	7C682D39
RC_9	392D687C	RC_{10}	B3A7E2F6
RC_{11}	A7B3F6E2	RC_{12}	8E9ADFCB

2 Design Principle of uHash

2.1 Parameters

The digest length h is jointly determined by the branch number b and block length n . An variant of uBlock with proven security foundation is adopted as the underlying block cipher, which sets the block length n to 256 bits. Correspondingly, the branch number b is configured as 2, 3 and 4 to produce digests of 512, 768 and 1024 bits respectively; in other words, b parallel block ciphers construct the compression function structure of uHash.

For uHash-512, uHash-768 and uHash-1024, the key length of the underlying block cipher is fixed to $4n$ (i.e., 1024 bits). Such the key is composed of $(b-1)n$ -bit chaining value and $(5-b)n$ -bit message, the length of message chunk processed by the compression function in each iteration is $(5-b)n$ bits, corresponding to 768, 512 and 256 bits respectively. The fixed-key-length configuration brings two merits: first, all variants of uHash share identical underlying block cipher implementation, lowering implementation overhead and facilitating unified design, analysis and evaluation; second, compared with relatively short-key constructions (e.g., Counter-2DM, Counter-3DM [2]), the compression functions of uHash-512 and uHash-768 process longer message, leading to a higher message processing rate (see Section 2.3 for details). The rationale for selecting other parameters of uBlock-1024 will be presented in Section 2.4.

2.2 Message Padding

Message padding is critical for the security of cryptographic hash functions. If different messages correspond to the same padded result, it would directly lead to collision attacks. The padding scheme of uHash is designed to guarantee that different messages will never yield the same padded result.

- Mandatory padding is applied to all input messages, with no unpadded case allowed. This prevents collisions between a message whose length is an integer multiple of an -bit, and another message after padding.
- The padded results of different messages are distinct. Let M'_1 and M'_2 be the padded results of messages M_1 and M_2 . If $M'_1 = M'_2$:
 - $M'_1 = M_1 \parallel 10^*$, $M'_2 = M_2 \parallel 10^*$. If $M'_1 = M'_2$, then $M_1 = M_2$.
 - $M'_1 = M_1 \parallel 1$, $M'_2 = M_2 \parallel 1$. If $M'_1 = M'_2$, then $M_1 = M_2$.

2.3 Overall Architecture

uHash is constructed based on block cipher. It processes message chunks through multi-branch parallel block ciphers, mapping an input message M of arbitrary length to a

digest H of optional length. The compression function adopts a variant of the Counter-bDM structure [2], where different branches are distinguished by counters. Part of the chaining value serves as the plaintext of the underlying block cipher, while the remaining chaining value and message chunks together form the master key of the block cipher, and the outputs of each branch form multi-block-length chaining values, achieving collision resistance and second-preimage resistance for the compression function. A feedback mechanism is introduced to provide the one-wayness of the compression function, ensuring preimage resistance of the algorithm. The branches are independent and can be executed completely in parallel, significantly improving message processing efficiency.

In the Counter-bDM compression function, the key length of the underlying block cipher is bn bits ($b = 2/3/4$). uHash adopts a variant Counter-bDM structure, fixing the key length of the block cipher to $4n$ bits. By extending the key length, the size of message chunks is increased, thereby optimizing the message processing rate (the number of blocks processed per call to the block cipher). For example, when the digest length is $2n$ bits, the message processing rate of uHash-512 can be three times that of Counter-2DM. In addition, compression functions of different scales reuse the same underlying block cipher, which can effectively reduce the software and hardware implementation costs of the uHash algorithm family.

Table 8: Message processing rate

	b	a	rate (a/b)
Counter-2DM	2	1	1/2
uHash-512	2	3	3/2
Counter-3DM	3	1	1/3
uHash-768	3	2	2/3
Counter-4DM	4	1	1/4
uHash-1024	4	1	1/4

2.4 Underlying Block Cipher

The underlying block cipher provides a key/message-controlled pseudorandom permutation (PRP) with block length of 256 bits and key length of 1024 bits. The encryption part adopts a variant of uBlock-256, while a new 1024-bit key schedule is designed. uBlock is the winner (ranked first overall) of the National Cryptographic Algorithm Design Competition held by the Chinese Association for Cryptologic Research (CACR) in 2019. Its security has undergone extensive public analysis and evaluation, and the performance has also been improved through various optimizations [3–11]. uBlock-1024 retains the main design features of uBlock-256, while performing targeted optimizations on the S-box, permutations, and key schedule.

The structure of uBlock-1024 follows the philosophy combining S-box and branch number, offering provable security against differential and linear cryptanalysis. It is also supported by mature analysis and evaluation theories against impossible differential cryptanalysis, integral cryptanalysis, meet-in-the-middle attacks, and other methods. Using the XOR operations and vector permutation instructions provided by the 256-bit registers in the SSE/AVX instruction sets, both the nonlinear transformation layer constructed from 4-bit S-boxes and the 4-bit-based vector permutations in the linear transformation can be implemented with a single instruction.

The 4-bit S-box can be efficiently implemented with SSE/AVX and other instruction sets, showing certain hardware implementation advantages over 8-bit S-boxes. The S-

box [12] of uBlock-1024 has no fixed points, with the maximum differential probability and the maximum linear probability achieving the optimal 2^{-2} , and the algebraic degree reaching the maximum of 3. It features a low latency of 0.12 ns and an area of 11.17 GE, which is significantly better than the original uBlock S-box with a latency of 0.16 ns and an area of 13.566 GE.

The linear transformation [13] of the diffusion layer adopts the optimal binary matrix of uBlock-256, where the 16×16 matrix has a branch number of 8, achieving the optimal value over $\text{GF}(2)$. The vector permutation uses a nibble-wise permutation, which requires fewer instructions than the original byte-wise permutation in uBlock-256. This reduces implementation cost while improving the security against differential cryptanalysis. Combined with the linear transformation, the full diffusion of the uBlock-1024 is achieved in 3 rounds, with a differential/linear branch number under super S-box of 4. Actually, the number of active S-boxes reaches 34 at 4 rounds and 51 at 5 rounds.

The key schedule of uHash-1024 is linear, making full use of easily implemented vector permutations. The subkeys are different permutations of the master key obtained through compositions of the f and g functions. Combined with round constants, this effectively avoids potential symmetry and periodicity, ensuring security against subspace attacks and weak-key attacks. The round constants are generated using an 8-bit linear feedback shift register, resulting in low hardware implementation cost.

3 Security Evaluation of uHash

3.1 Security Claims

Under the ideal cipher model, uHash admits provable security in terms of collision resistance, preimage resistance, and second-preimage resistance. Additionally, its output digests are computationally indistinguishable from uniformly random bitstrings of the same length.

Table 9: Security claims of uHash

Algorithm	Digest length	Security (bits)			
		Coll	Pre	SecPre	Indist
uHash-512	512	256	512	512	256
uHash-768	768	384	768	768	384
uHash-1024	1024	512	1024	1024	512

3.2 Security Proof

The underlying block cipher uBlock-1024 (denoted as E for brevity) is regarded as a family of random permutations indexed by the key. One call to E or E^{-1} is computationally equivalent to one query, so the security of the algorithm is measured by the number of queries q required by the attack.

To analyze the security of the uHash algorithm, the query results are recorded using a combination of databases and graphs. The construction methods of the databases and the symbolic definitions used in the proof are as follows.

Databases $\mathcal{D}_H$ and $\mathcal{D}_E$ The adversary can make three types of queries: query uHash with a message M to obtain the corresponding digest H , query the underlying block cipher E with (X, K) to obtain the corresponding ciphertext Y , or query the inverse operation

E^{-1} with (Y, K) to obtain the corresponding plaintext X . Let databases $\mathcal{D}_H$ and $\mathcal{D}_E$ be established, where the query record (M, H) of uHash is logged into database $\mathcal{D}_H$, and the query records (X, K, Y) of E and E^{-1} are logged into database $\mathcal{D}_E$.

Extended Queries Note that each state u in uHash is of bn bits, and its collisions and preimages are based on bn -bit states/images. To facilitate the estimation of the adversary's capabilities, its query capabilities are appropriately amplified: let $c = 2^{\lceil \log_2 b \rceil}$ (i.e., there exists an integer λ such that $2^{\lambda-1} < b \leq 2^\lambda = c$). Each query to E or E^{-1} by the adversary can be extended to c mappings under K . Specifically:

- $E(X, K)$:
 - First, set $X_1 = X$ and compute $Y_1 = E(X_1, K)$;
 - Then, for $j = 2, \dots, c$, let $X_j = X_1 \oplus (j-1)$, and compute the corresponding $Y_j = E(X_j, K)$;
 - Log $\{(X_j, K, Y_j)\}_{j \in [1, c]}$ into database $\mathcal{D}_E$.
- $E^{-1}(Y, K)$:
 - First, set $Y_1 = Y$ and compute $X_1 = E^{-1}(Y_1, K)$;
 - Then, for $j = 2, \dots, c$, let $X_j = X_1 \oplus (j-1)$, and compute the corresponding $Y_j = E(X_j, K)$;
 - Log $\{(X_j, K, Y_j)\}_{j \in [1, c]}$ into database $\mathcal{D}_E$.

The mapping from X to Y under $E(\cdot, K)$ is denoted as $X \xrightarrow{K} Y$, and the extended c mappings are denoted as $\mathbf{X} \xrightarrow{K} \mathbf{Y}$.

Graphs and Paths The state of uHash is abbreviated as $u_i = u_i^1 \parallel u_i^*$, where $u_i^* = u_i^2 \parallel u_i^3 \parallel \dots \parallel u_i^b$ forms the key together with the message chunk M_i , i.e. $K_i = u_i^* \parallel M_i$ (for a query-specified key K_i , we write $K_i = K_i^* \parallel K_i^M \in \{0, 1\}^{(b-1)n} \times \{0, 1\}^{an}$). Treating each state u_i as a vertex in graph $\mathcal{T}$, the round function iteration can be described as a path from u_i to u_{i+1} determined by message M_i , denoted as $u_i \xrightarrow{M_i} u_{i+1}$. Clearly, the digest H of message M corresponds to a path starting from the initial vector IV :

$$IV = u_1 \xrightarrow{M_1} u_2 \xrightarrow{M_2} \dots \xrightarrow{M_{l-1}} u_l \xrightarrow{M_l} u_{l+1} = H,$$

where the (padded) message $M = M_1 \parallel M_2 \parallel \dots \parallel M_l$. Due to the different final iteration operation, the step $u_l \xrightarrow{M_l} u_{l+1}$ is not included in graph $\mathcal{T}$; only the b mappings from this step are logged into database $\mathcal{D}_E$.

By the definition of uHash, a valid hash function query corresponds to a tree-like path in graph $\mathcal{T}$ starting from the initial vector IV , where each node is a bn -bit state.

Furthermore, we give a supplementary definition for queries to E :

For a query $E(X, K)$, if there exists a vertex $u_z = u_z^1 \parallel u_z^*$ in graph $\mathcal{T}$ and some $j \in [1, b]$ satisfying

$$\begin{cases} u_z^1 \oplus (j-1) = X \\ u_z^* = K^* \end{cases}$$

then take u_z as the starting point, use K^M as the message chunk, and compute the path $u_z \xrightarrow{K^M} u_{z+1}$ according to the iteration rule. This path is added to graph $\mathcal{T}$, and the corresponding b mappings are logged into database $\mathcal{D}_E$.

If no such vertex exists in graph $\mathcal{T}$, then compute c mappings according to the extended query rule and log them into database $\mathcal{D}_E$.

Observation: For a randomly chosen query $E(X, K)$ or $E^{-1}(Y, K)$, the probability that its result falls in graph $\mathcal{T}$ is at most $c|\mathcal{T}|/2^{bn}$, where $|\mathcal{T}|$ denotes the total number of vertices in graph $\mathcal{T}$.

Based on the construction of the databases and graph, the attack can be formally described, and the success probability of the attack can be further estimated.

3.2.1 Preimage Security

For a target image $H \in \{0, 1\}^{bn}$, a preimage attack means that the adversary $\mathcal{A}$ successfully finds a complete path from the initial vector IV to the target image H . According to the definitions of the graph and databases in this chapter, that is, after q queries:

PRE: There exists a path $IV \xrightarrow{M} u_z$ in graph $\mathcal{T}$, and all mappings of $u_z \xrightarrow{M_z} H$ exist in database $\mathcal{D}_E$.

First, consider the final iteration. For the target image H , the probability of obtaining a tuple (X, K, Y) by querying E such that $Y = H^k \oplus X \oplus (k - 1)$ (for some $k \in [1, b]$) is at most $b/2^n$. The probability of further satisfying the system of equations $E(X \oplus (k - 1) \oplus (j - 1), K \oplus (k - 1) \oplus 16 \oplus (j - 1)) \oplus X \oplus (k - 1) = H^j$ for all $j \in [1, b]$ is at most $1/2^{(b-1)n}$. In other words, the probability of obtaining a valid final iteration leading to the target image H by querying E is at most $b/2^{bn}$.

Similarly, the probability of obtaining a tuple (X, K, Y) by querying E^{-1} such that $Y = H^k \oplus X \oplus (k - 1)$ (for some $k \in [1, b]$) is also at most $b/2^n$; the probability of further obtaining a valid final iteration satisfying H is also at most $b/2^{bn}$.

Consider the i -th query, and assume that no complete path satisfying the preimage attack has been found so far. If the query is to $E(X, K)$, then there exists a vertex $u = u^1 \parallel u^*$ in graph $\mathcal{T}$ and some $j \in [1, b]$ such that

$$\begin{cases} u^1 \oplus (j - 1) = X \\ u^* = K^* \end{cases}$$

A preimage can only be formed if this query happens to yield the b equations of the final iteration $u_z \xrightarrow{M_z} H$, and the probability of this event is at most $1/2^{bn}$. If the query is to $E^{-1}(Y, K)$, then the probability of finding a complete forward path (starting from IV) is at most the probability of finding a valid predecessor point u satisfying $u \xrightarrow{K^M} u_z \xrightarrow{M_z} H$, where $Y = u_z^j$ ($j \in [1, b]$). That is, the success probability is at most $b/2^{bn}$.

Therefore,

$$\Pr[\text{PRE}(q)] \leq \sum_{i=1}^q \frac{b}{2^{bn}} = \frac{bq}{2^{bn}}.$$

3.2.2 Second Preimage Security

For any preimage M and its target image $H = \text{uHash}(M)$, there already exists a complete path in graph $\mathcal{T}$ from the initial vector IV to H :

$$IV = u_1 \xrightarrow{M_1} u_2 \xrightarrow{M_2} \cdots \xrightarrow{M_{l-1}} u_l \xrightarrow{M_l} u_{l+1} = H.$$

A second preimage attack means that the adversary $\mathcal{A}$ successfully finds another complete path from the initial vector IV to the target image H , i.e.,

SEC: There exists a path $IV \xrightarrow{M'} u_z$ in graph $\mathcal{T}$, and all mappings of $u_z \xrightarrow{M_z} H$ exist in database $\mathcal{D}_E$, where $M' \parallel M_z \neq M$.

The adversary $\mathcal{A}$ for a second preimage attack has two ways to succeed: (1) finding another preimage of H , or (2) finding a path to any point u_z on the path from IV to H . We define the latter event as INCOLL.

INCOLL: There exists a path $IV \xrightarrow{M'} u_z$ in graph $\mathcal{T}$, where $u_z \in \{u_1, \dots, u_l\}$.

Consider the i -th query, and assume that no path from IV to any path point $u_z \in \{u_1, \dots, u_l\}$ has been found so far. If the query is to $E(X, K)$, then there must exist a vertex $u = u^1 \parallel u^*$ in graph $\mathcal{T}$ and some $j \in [1, b]$ such that

$$\begin{cases} u^1 \oplus (j-1) = X \\ u^* = K^* \end{cases}$$

According to the definitions, the probability of obtaining an extended path $u \xrightarrow{K^M} u'$ with $u' \in \{u_1, \dots, u_l\}$ through this query is at most $l/2^{bn}$.

If the query is to $E^{-1}(Y, K)$, to connect a vertex $u = u^1 \parallel u^*$ in graph $\mathcal{T}$ with some $u_z \in \{u_1, \dots, u_l\}$, we must have $K^* = u^*$ and $Y = u^1 \oplus u_z^j$ (we may also take $Y = u^1 \oplus u_z^j \oplus (j-1)$, which yields an equivalent result). The probability that all b equations hold is at most $1/2^{bn}$.

If we consider establishing a connection between two path segments $IV \xrightarrow{M'} u$ and $u' \xrightarrow{M'_2} u_z$, the success probability is no higher than that of directly establishing a connection between u and u_z . Therefore,

$$\begin{aligned} \Pr[\text{SEC}(q)] &\leq \Pr[\text{PRE}(q)] + \Pr[\text{INCOLL}(q)] \\ &\leq \sum_{i=1}^q \frac{b}{2^{bn}} + \sum_{i=1}^q \frac{l}{2^{bn}} \\ &\leq \frac{(b+l)q}{2^{bn}}. \end{aligned}$$

3.2.3 Collision Security

A collision attack means that the adversary $\mathcal{A}$ successfully finds two distinct complete paths from the initial vector IV to the same digest H . According to the definitions of the graph and databases, the condition for a collision attack can be relaxed: the attack is considered successful if the adversary finds two distinct paths from IV to the same state node. That is, after q queries,

COLL: There exist two paths $IV \xrightarrow{M} u_z$ and $IV \xrightarrow{M'} u_z$ in graph $\mathcal{T}$, with $M \neq M'$.

First, consider the case where the collision occurs at a state node $u_z \in \{0, 1\}^{bn}$ that is not the end of a path. Consider the i -th query, and assume that no collision satisfying the conditions has been found so far. If the query is to $E(X, K)$, then there must exist a vertex $u = u^1 \parallel u^*$ in graph $\mathcal{T}$ and some $j \in [1, b]$ such that

$$\begin{cases} u^1 \oplus (j-1) = X \\ u^* = K^* \end{cases}$$

According to the extension rule, this query can be extended to b mappings

$$E(u^1 \oplus (j-1), K) = Y_j, \quad j = 1, \dots, b.$$

Compute $u_{zz}^j = Y_j \oplus u^1$ ($j = 1, \dots, b$), yielding the extended path $u \xrightarrow{K^M} u_{zz}$. The probability that the new node u_{zz} collides with some node u_z ($u_z \in \mathcal{T}$) in the graph is at most $|\mathcal{T}|/2^{bn}$.

If the query is to $E^{-1}(Y, K)$, to connect a vertex $u = u^1 \parallel u^*$ in graph $\mathcal{T}$ with some node u_z , we must have $K^* = u^*$ and $Y = u^1 \oplus u_z^j$ (we may also take $Y = u^1 \oplus u_z^j \oplus (j-1)$, which yields an equivalent result). The probability that all b equations hold is at most $1/2^{bn}$.

If the state node is the end of at least one of the paths, i.e., " $\dots \xrightarrow{M_l} u_z$ ", then the b equations are set according to the final iteration. The collision probability is also at most $|\mathcal{T}|/2^{bn}$. Therefore,

$$\Pr[\text{COLL}(q)] \leq \sum_{i=1}^q \frac{i}{2^{bn}} \leq \frac{q^2}{2^{bn+1}}.$$

3.2.4 Indistinguishability

To characterize the distinguishability between uHash and a random function, we construct a simulator $\mathcal{S}/\mathcal{S}^{-1}$. The simulator simulates a family of random permutations π/π^{-1} by invoking a random oracle and query records. Meanwhile, we build a database $\mathcal{D}_S$ to store all query records of $\mathcal{S}/\mathcal{S}^{-1}$, and a graph $\mathcal{T}$ to record all paths starting from the initial vector IV .

To simulate the random permutation family π/π^{-1} , we adopt a sampling without replacement method. The supplementary definitions are as follows:

$$\mathcal{D}_{S(\cdot, K)} = \{Y \mid \exists X, \text{ s.t. } (X, K, Y) \in \mathcal{D}_S\}$$

$$\mathcal{D}_{S(K, \cdot)} = \{X \mid \exists Y, \text{ s.t. } (X, K, Y) \in \mathcal{D}_S\}$$

The simulator is defined as Algorithm 2, and $c = 2^{\lceil \log_2 b \rceil}$.

For queries whose path corresponds to the final iteration in the graph, the simulator $\mathcal{S}$ obtains the digest value H by calling the random oracle $\mathcal{R}$, and then computes b mappings in reverse (Case 1); For queries whose path corresponds to the graph (but not the final iteration), the simulator $\mathcal{S}$ simulates the random permutation $\pi(\cdot, \cdot)$ via random sampling and records the new path (Case 2); For randomly chosen inputs that do not correspond to any path in the graph, the simulator $\mathcal{S}$ or $\mathcal{S}^{-1}$ generates and records c mappings via random sampling (Cases 3 and 4). In particular, if the X obtained by random sampling in $\mathcal{S}^{-1}$ is such that $(X \oplus (j-1), K^*)$ (for some $j \in [1, c]$) is a point on some path in the graph, then this query constitutes a path extension, and the new path is recorded.

To record the construction process of the graph $\mathcal{T}$, let $\mathcal{T}^{(i)}$ denote the graph after the i -th query. $\mathcal{T}^{(0)}$ is the initial state, containing only the point corresponding to the initial value IV .

According to the previous definitions, $u_z \xrightarrow{M} u_{zz} \subseteq \mathcal{T}$ denotes that there exists a path in $\mathcal{T}$ "starting from u_z , determined by M , and reaching u_{zz} ", which is called a "complete path", with the following relations:

$$\left\{ \begin{array}{l} u_z^1 \xrightarrow{u_z^* \parallel M} Y^1 \xrightarrow{\oplus u_z^1} u_{zz}^1 \\ u_z^1 \oplus 1 \xrightarrow{u_z^* \parallel M} Y^2 \xrightarrow{\oplus u_z^1} u_{zz}^2 \\ \vdots \\ u_z^1 \oplus (b-1) \xrightarrow{u_z^* \parallel M} Y^b \xrightarrow{\oplus u_z^1} u_{zz}^b \end{array} \right.$$

where $X \xrightarrow{K} Y$ denotes a mapping defined by E_K ; and $\xrightarrow{\oplus X}$ is defined as $Y \xrightarrow{\oplus X} Y \oplus X$.

Algorithm 2 Simulator $\mathcal{S}/\mathcal{S}^{-1}$

```

1: Function  $\mathcal{S}(X, K)$ 
2: if  $\exists(X, K, Y) \in \mathcal{D}_S$  then
3:   return  $Y$ 
4: end if
5: if  $\exists IV \stackrel{M}{\Rightarrow} (u^1, u^*) \in \mathcal{T}$  and  $j \in [1, b]$  ▷ Check for path ending at this point
6:   s.t.  $\begin{cases} u^1 \oplus (j-1) = X \\ u^* \oplus (j-1) = K^* \end{cases}$  then
7:     Query  $\mathcal{R}$  with  $M' \leftarrow M \parallel \text{unpad}(K^M)$ 
8:      $H \leftarrow \mathcal{R}(M')$ 
9:     return  $H^j \oplus u^1$ 
10:   for  $i = 1, \dots, b$ , do
11:      $\mathcal{D}_S \leftarrow \mathcal{D}_S \cup \{(u^1 \oplus (i-1), K \oplus 16 \oplus (i-1), H^i \oplus u^1)\}$ 
12:   end for
13: end if
14: if  $\exists IV \stackrel{M}{\Rightarrow} (u^1, u^*) \in \mathcal{T}$  and  $j \in [1, b]$  ▷ Check for path passing through this point
15:   s.t.  $\begin{cases} u^1 \oplus (j-1) = X \\ u^* = K^* \end{cases}$  then
16:     for  $i = 1, \dots, c$ , do
17:        $Y^i \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{S(\cdot, K)}$  ▷ Sampling without replacement
18:        $\mathcal{D}_S \leftarrow \mathcal{D}_S \cup \{(u^1 \oplus (i-1), K, Y^i)\}$ 
19:     end for
20:     return  $Y^j$ 
21:   Add  $(u^1, u^*) \stackrel{K^M}{\Rightarrow} (Y^1 \oplus u^1, \dots, Y^b \oplus u^1)$  to  $\mathcal{T}$ 
22: else
23:   for  $i = 1, \dots, c$  do
24:      $Y^i \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{S(\cdot, K)}$ 
25:      $\mathcal{D}_S \leftarrow \mathcal{D}_S \cup \{(X \oplus (i-1), K, Y^i)\}$ 
26:   end for
27:   return  $Y^1$ 
28: end if
29: End Function

30: Function  $\mathcal{S}^{-1}(Y, K)$ 
31: if  $\exists(X, K, Y) \in \mathcal{D}$  then
32:   return  $X$ 
33: else
34:   return  $X \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{S(K, \cdot)}$ 
35:    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(X, K, Y)\}$ 
36:   for  $i = 2, \dots, c$  do
37:     ▷ If  $X \oplus (i-1)$  already has an assignment in  $\mathcal{D}_{S(K, \cdot)}$ , keep the original value
38:     if  $X \oplus (i-1) \notin \mathcal{D}_{S(K, \cdot)}$  then
39:        $Y^i \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{S(\cdot, K)}$ 
40:        $\mathcal{D} \leftarrow \mathcal{D} \cup \{(X \oplus (i-1), K, Y^i)\}$ 
41:     end if
42:   end for
43:   ▷ If  $(X \oplus (j-1), K^*)$  is a point on some path, extend the path with assigned values
44:   if  $\exists(u^1, u^*) \in \mathcal{T}$  and  $j \in [1, c]$ 
45:     s.t.  $\begin{cases} u^1 = X \oplus (j-1) \\ u^* = K^* \end{cases}$  then
46:       Add  $(u^1, u^*) \stackrel{K^M}{\Rightarrow} (Y^j \oplus u^1, \dots, Y^{j+b-1} \oplus u^1)$  to  $\mathcal{T}$ 
47:     end if
48: end if
49: End Function

```

Correspondingly, the c mappings generated by random inputs (X, K) or (Y, K) are called “incomplete paths”, with the following relations:

$$\left\{ \begin{array}{l} X \xrightarrow{K} Y^1 \\ X \oplus 1 \xrightarrow{K} Y^2 \\ \vdots \\ X \oplus (c-1) \xrightarrow{K} Y^c \end{array} \right.$$

The starting point of such a path can be $(X \oplus (i-1), K^*)$ (for $i \in [1, c]$, where $K = K^* \parallel K^M$), and the end point is $(Y^i, \dots, Y^{(i+b-1) \bmod c})$.

According to the construction of the graph $\mathcal{T}$, $\mathcal{T}$ contains all complete paths, and every point in the graph lies on a complete path chain starting from IV .

We define the following three BAD events. It can be shown that, conditioned on no BAD event occurring, $(\mathcal{R}, \mathcal{S}[\mathcal{R}]^\pm)$ and $(\text{uHash}[\pi], \pi^\pm)$ are indistinguishable.

- **CONNECT**: The incomplete path generated by a randomly chosen input (which does not correspond to any path in the graph) connects with a path in the graph.

$$\exists \{(X, K, Y^1), (X \oplus 1, K, Y^2), \dots, (X \oplus (c-1), K, Y^c)\} \subseteq \mathcal{D}_S, \text{ s.t.}$$

$$(X, K^*) \in \mathcal{T} \wedge (Y^1 \oplus X, \dots, Y^b \oplus X) \notin \mathcal{T}, \text{ or } (X, K^*) \notin \mathcal{T} \wedge (Y^1 \oplus X, \dots, Y^b \oplus X) \in \mathcal{T}$$

- **MCOLL**: Collision between different paths in the graph.

$$\exists M, M', \text{ s.t. } M \neq M', \text{ and } IV \xrightarrow{M} u_z, IV \xrightarrow{M'} u_z \subset \mathcal{T}$$

In other words,

$$\exists u_z, u_{zz}, u \in \mathcal{T}, u_z \neq u_{zz}, \text{ and } M_z, M_{zz} \in \{0, 1\}^{an} \text{ s.t.}$$

$$u_z \xrightarrow{M_z} u, u_{zz} \xrightarrow{M_{zz}} u \subseteq \mathcal{T}$$

- **FALSE**: The behavior of $\mathcal{S}/\mathcal{S}^{-1}$ does not satisfy the permutation property.

$$\exists (X, K, Y), (X', K, Y') \in \mathcal{D}_S, \text{ s.t. } X \neq X' \wedge Y = Y' \text{ or } X = X' \wedge Y \neq Y'$$

In other words, for the same K , if different records in the database have the same X or the same Y , then $\mathcal{S}(\cdot, K)/\mathcal{S}^{-1}(\cdot, K)$ is not a permutation. Specifically, FALSE can be divided into two cases:

- **RESET**: Under $\mathcal{S}(\cdot, K)/\mathcal{S}^{-1}(\cdot, K)$, the same X is assigned different values.

$$\exists (X, K, Y), (X', K, Y') \in \mathcal{D}_S, \text{ s.t. } X = X' \wedge Y \neq Y'$$

- **FCOLL**: Under $\mathcal{S}(\cdot, K)/\mathcal{S}^{-1}(\cdot, K)$, different X, X' are mapped to the same Y .

$$\exists (X, K, Y), (X', K, Y') \in \mathcal{D}_S, \text{ s.t. } X \neq X' \wedge Y = Y'$$

Assume that no BAD events have occurred in the first $i-1$ queries. We consider the probability of several events occurring at the i -th query. To clearly classify possible cases, we use the previously defined Cases 1, 2, 3, and 4 to describe the type of the i -th query and the behavior of the simulator $\mathcal{S}$ or $\mathcal{S}^{-1}$.

CONNECT

The CONNECT event considers the definition where an incomplete path connects to a complete path in the graph, so this event may occur when the query falls into Cases 2, 3, or 4.

When the i -th query is in Case 2, for a query $\mathcal{S}(X, K)$, we can find $(u^1, u^*) \in \mathcal{T}$ and $j \in [1, b]$ such that

$$\begin{cases} u^1 = X \oplus (j - 1), \\ u^* = K^* \end{cases}$$

The simulator $\mathcal{S}$ obtains a set of Y^i via sampling: $Y^i \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{\mathcal{S}(\cdot, K)}$ ($i = 1, \dots, c$). Let $\tilde{X} = X \oplus (j - 1)$. The extended path can be described as follows:

$$\begin{cases} \tilde{X} \xrightarrow{K} Y^1 \xrightarrow{\oplus \tilde{X}} u_{zz}^1 \\ \tilde{X} \oplus 1 \xrightarrow{K} Y^2 \xrightarrow{\oplus \tilde{X}} u_{zz}^2 \\ \vdots \\ \tilde{X} \oplus (b - 1) \xrightarrow{K} Y^b \xrightarrow{\oplus \tilde{X}} u_{zz}^b \end{cases}$$

Then, the occurrence of the CONNECT event means that $(Y^1 \oplus \tilde{X}, Y^2 \oplus \tilde{X}, \dots, Y^b \oplus \tilde{X})$ falls on an incomplete path. Formally,

If it collides with the starting point of an incomplete path, i.e.,

$$\exists K' \in \{0, 1\}^{an}, Y_{zz}^1, \dots, Y_{zz}^c \in \{0, 1\}^n, \text{ s.t.}$$

$$\begin{cases} (Y^1 \oplus \tilde{X}, Y^2 \oplus \tilde{X} \parallel \dots \parallel Y^b \oplus \tilde{X} \parallel K', Y_{zz}^1) \in \mathcal{D}_{\mathcal{S}} \\ (Y^1 \oplus \tilde{X} \oplus 1, Y^2 \oplus \tilde{X} \parallel \dots \parallel Y^b \oplus \tilde{X} \parallel K', Y_{zz}^2) \in \mathcal{D}_{\mathcal{S}} \\ \vdots \\ (Y^1 \oplus \tilde{X} \oplus (c - 1), Y^2 \oplus \tilde{X} \parallel \dots \parallel Y^b \oplus \tilde{X} \parallel K', Y_{zz}^c) \in \mathcal{D}_{\mathcal{S}} \end{cases}$$

If it collides with the end point of an incomplete path, i.e.,

$$\exists (X', K, Y'^1), (X' \oplus 1, K, Y'^2), \dots, (X' \oplus (c - 1), K, Y'^c) \in \mathcal{D}_{\mathcal{S}}, \text{ s.t.}$$

$$\begin{cases} Y'^1 \oplus X' = Y^1 \oplus \tilde{X} \\ Y'^2 \oplus X' = Y^2 \oplus \tilde{X} \\ \vdots \\ Y'^b \oplus X' = Y^b \oplus \tilde{X} \end{cases}$$

Note that the previous $i - 1$ queries generate at most $i - 1$ sets of incomplete paths, each containing c paths with different starting points. Since the (end) point of the path generated by the i -th query is determined, namely $(Y^1 \oplus \tilde{X}, Y^2 \oplus \tilde{X}, \dots, Y^b \oplus \tilde{X})$, the probability that this point coincides with the starting point of an incomplete path in $\mathcal{D}_{\mathcal{S}}$ is at most $c(i - 1)/2^{bn}$. On the other hand, the $i - 1$ sets of incomplete paths in $\mathcal{D}_{\mathcal{S}}$ can form at most $c(i - 1)$ end points, so the probability that $(Y^1 \oplus \tilde{X}, Y^2 \oplus \tilde{X}, \dots, Y^b \oplus \tilde{X})$ collides with one of these end points is at most $c(i - 1)/2^{bn}$.

When the i -th query is in Case 3, $\mathcal{S}(X, K)$ generates c mappings

$$\begin{cases} X \xrightarrow{K} Y \\ X \oplus 1 \xrightarrow{K} Y^2 \\ \vdots \\ X \oplus (c - 1) \xrightarrow{K} Y^c \end{cases}$$

Thus, c paths with different start/end points can be formed.

The occurrence of the CONNECT event means: $\exists u \in \mathcal{T}, j \in [1, c]$, s.t.

$$\left\{ \begin{array}{l} Y^j \oplus (X \oplus (j-1)) = u^1 \\ Y^{j+1} \oplus (X \oplus j \bmod c) = u^2 \\ \vdots \\ Y^{(j+b-1) \bmod c} \oplus (X \oplus (j+b-2) \bmod c) = u^b \end{array} \right.$$

That is, the end point of the incomplete path happens to lie on the graph. For the i -th query, there are at most $i-1$ points in the graph $\mathcal{T}$ (excluding IV). Therefore, the probability that any of the c possible incomplete path end points collides with at most i points on the graph is at most $c(i-1)/2^{bn}$.

When the i -th query is in Case 4 and does not generate path extension, $\mathcal{S}^{-1}(Y, K)$ generates c mappings. At this point, the probability analysis of the CONNECT event is similar to that of Case 3.

Thus, we have

$$\Pr[\text{connect}(i)] \leq c(i-1)/2^{bn},$$

$$\Pr[\text{CONNECT}(q)] \leq \sum_{i=1}^q c(i-1)/2^{bn} \leq cq(q-1)/2^{bn+1}.$$

Note: When the end point of an incomplete path connects to a path in the graph, since there is no complete path starting from IV , this case does not directly threaten security. For simplicity of analysis, this case is included in the BAD event. It can be seen from the analysis that this does not affect the probability of the BAD event occurring.

MCOLL

According to the definition of the MCOLL event, only path collisions on the graph $\mathcal{T}$ are considered, so we only need to consider the probability of the MCOLL event occurring when path extension is generated in Case 2 and Case 4.

When the i -th query is in Case 2, the simulator $\mathcal{S}(X, K)$ generates an extended path

$$\left\{ \begin{array}{l} \tilde{X} \xrightarrow{K} Y^1 \xrightarrow{\oplus \tilde{X}} u_{zz}^1 \\ \tilde{X} \oplus 1 \xrightarrow{K} Y^2 \xrightarrow{\oplus \tilde{X}} u_{zz}^2 \\ \vdots \\ \tilde{X} \oplus (b-1) \xrightarrow{K} Y^b \xrightarrow{\oplus \tilde{X}} u_{zz}^b \end{array} \right.$$

Then, the occurrence of the MCOLL event means that

$$(Y^1 \oplus \tilde{X}, Y^2 \oplus \tilde{X}, \dots, Y^b \oplus \tilde{X}) \in \mathcal{T}_{i-1} - \{IV\}.$$

Note that the previous $i-1$ queries make at most i points exist in the graph $\mathcal{T}$ (including IV). Therefore, combined with the randomness of the sampling of Y^i , the probability that MCOLL occurs at this time is at most $(i-1)/2^{bn}$.

When the i -th query is in Case 4 and path extension occurs, $\mathcal{S}^{-1}(Y, K)$ generates c mappings

$$\left\{ \begin{array}{l} X \xrightarrow{K} Y \\ X \oplus 1 \xrightarrow{K} Y^2 \\ \vdots \\ X \oplus (c-1) \xrightarrow{K} Y^c \end{array} \right.$$

We can query $(u^1, u^*) \in \mathcal{T}$ and $j \in [1, b]$ such that

$$\begin{cases} u^1 = X \oplus (j-1), \\ u^* = K^* \end{cases}$$

Similarly, let $\tilde{X} = X \oplus (j-1)$. Note that when an extended path is queried, $\tilde{X}$ (and thus j) is determined. Therefore, similar to Case 2, the probability that MCOLL occurs at this time is at most $(i-1)/2^{bn}$.

$$\Pr[\text{mcoll}(i)] \leq (i-1)/2^{bn},$$

$$\Pr[\text{MCOLL}(q)] \leq \sum_{i=1}^q (i-1)/2^{bn} \leq q(q-1)/2^{bn+1}.$$

FALSE

When the i -th query is in Case 1, for a query $\mathcal{S}(X, K)$, we can find $(u^1, u^*) \in \mathcal{T}$ and $j \in [1, b]$ such that

$$\begin{cases} u^1 = X \oplus (j-1) \\ u^* = K^* \oplus (j-1) \end{cases}$$

Let $\tilde{X} = X \oplus (j-1)$, $\tilde{K}^* = K^* \oplus (j-1)$, and $\tilde{K} = \tilde{K}^* \parallel K^M$. The simulator $\mathcal{S}$ accesses $\mathcal{R}$ to obtain a random output H , and computes a set of $Y^i \leftarrow H^i \oplus \tilde{X}$ ($i = 1, \dots, b$) in reverse, assigning the following b mappings:

$$\left\{ \begin{array}{l} \tilde{X} \xrightarrow{\tilde{K}^* \parallel K^M} Y^1 \\ \tilde{X} \oplus 1 \xrightarrow{\tilde{K}^* \oplus 1 \parallel K^M} Y^2 \\ \vdots \\ \tilde{X} \oplus (b-1) \xrightarrow{\tilde{K}^* \oplus (b-1) \parallel K^M} Y^b \end{array} \right.$$

For queries in Case 1, the choice of $(\tilde{X}, \tilde{K}^*)$ is determined by the points in the graph, with no more than i choices, while K^M can be chosen freely.

Observe that $((\tilde{X}, \tilde{K}^*), (\tilde{X} \oplus 1, \tilde{K}^* \oplus 1), \dots, (\tilde{X} \oplus (c-1), \tilde{K}^* \oplus (c-1)))$ forms a cycle.

If $\tilde{K}^* \oplus (j-1)$ (where $j \in [1, c]$) has been queried, without loss of generality, assume that $\tilde{K} = \tilde{K}^* \oplus (j-1) \parallel K^M$ has been queried.

- If $\tilde{X} \oplus (j-1) \in \mathcal{D}_{\mathcal{S}(K, \cdot)}$, then the RESET event occurs. The previous $i-1$ queries make $\mathcal{D}_{\mathcal{S}(K, \cdot)}$ contain at most $c(i-1)$ values, so the probability that the RESET event occurs in this case is at most $|\mathcal{D}_{\mathcal{S}(K, \cdot)}|/2^{bn} \leq c(i-1)/2^{bn}$.

- If $\tilde{X} \oplus (j-1) \notin \mathcal{D}_{\mathcal{S}(K, \cdot)}$, then assigning Y^i according to the simulation rule may cause the FCOLL event to occur. Similarly, the previous $i-1$ queries make $\mathcal{D}_{\mathcal{S}(\cdot, K)}$ contain at most $c(i-1)$ values, so the probability that the FCOLL event occurs in this case is at most $|\mathcal{D}_{\mathcal{S}(\cdot, K)}|/2^{bn} \leq c(i-1)/2^{bn}$.

If the number of different queries for the same K exceeds $2^n/c$, according to the simulation rules, it can be considered that all 2^n mappings under K have been obtained.

If $\tilde{K}^* \oplus (j-1)$ has been queried as the end of a path (i.e., Case 1), then there exists another path end point (u_z^1, u_z^*) such that $u_z^* \oplus (\theta-1) = \tilde{K}^* \oplus (j-1)$ for some $\theta \in [1, c]$. Since the points on the path in the graph are generated by random sampling, the probability of this case is at most $1/2^{(b-1)n - (\log_2 c + 1)}$. Furthermore, the probability that the RESET or FCOLL event occurs is at most $(i-1)/2^{bn - (\log_2 c + 1)}$.

If for all $j \in [1, c]$, $\widetilde{K}^* \oplus (j-1)$ has not been queried, then the probability that the RESET or FCOLL event occurs is 0.

When the i -th query is in Case 2, similar to Case 1, the choice of $(\tilde{X}, K^*)$ is determined by the points in the graph, with no more than i choices, while K^M can be chosen freely.

In this case, since Y^i is sampled without replacement, the probability that the FCOLL event occurs is 0. Next, we consider the probability that the RESET event occurs.

If K collides with the key $\widetilde{K}^{*+} \oplus (j'-1) \parallel K^{M+}$ in the record $(\tilde{X}^+ \oplus (j'-1), \widetilde{K}^{*+} \oplus (j'-1) \parallel K^{M+}, Y^{j'+})$ previously generated by queries in Case 1. At this point, $u = (\tilde{X}, K^*)$ is an intermediate point on some path in the graph, and $u^+ = (\tilde{X}^+, \widetilde{K}^{*+})$ is the end of some path in the graph, and $\exists j \in [1, b]$ s.t. $K^* = \widetilde{K}^{*+} \oplus (j'-1)$. Since the points on the graph are generated by random sampling, and there are at most $i-1$ points u^+ satisfying the conditions on the graph, the probability of this case is at most $(i-1)/2^{(b-1)n}$. At this point, if there exists an element in $\{\tilde{X} \oplus (j-1)\}_{j \in [1, c]}$ that collides with $\tilde{X}^+ \oplus (j'-1)$, then the RESET event occurs with probability at most $c/2^n$. Therefore, if K collides with a record previously generated by queries in Case 1, the probability that RESET occurs is at most $c(i-1)/2^{bn}$.

If K collides with a record previously generated by queries in Case 2, then there exist at least two distinct points $u = (\tilde{X}, K^*)$ and $u^+ = (\tilde{X}^+, K^*)$ on the graph. The probability of this case is at most $(i-1)/2^{(b-1)n}$. If $\exists j \in [1, c]$ s.t. $\tilde{X} = \tilde{X}^+ \oplus (j-1)$, then X should already be assigned (i.e., $(X, K, \cdot) \in \mathcal{D}_S$), so there is no need to resample and assign, meaning the premise of Case 2 does not hold; if $\{u \oplus (j-1)\}_{j \in [1, c]} \cap \{u^+ \oplus (j-1)\}_{j \in [1, c]} = \emptyset$, then according to the without-replacement sampling rule, the RESET event does not occur at this time.

If K collides with a record previously generated by queries in Case 3 or 4, it means that this query connects the incomplete path generated by previous Case 3/4 queries to a path in the graph, i.e., the CONNECT event.

Therefore, when the i -th query is in Case 2, the probability that the FALSE event occurs is at most $c(i-1)/2^{bn}$.

When the i -th query is in Case 3, it means that (X, K^*) does not satisfy the conditions of Case 1 or 2, so it cannot collide with records previously generated by queries in Case 1 or 2.

If K collides with a record $\{(X^+ \oplus (j'-1), K, Y^{j'+})\}_{j' \in [1, c]} \subseteq \mathcal{D}_S$ previously generated by queries in Case 3, there are two cases: if $X \in \{(X^+ \oplus (j'-1), K, Y^{j'+})\}_{j' \in [1, c]} \in \mathcal{D}_S$, then it should already be assigned, and no simulation sampling is required; if $X \notin \{(X^+ \oplus (j'-1), K, Y^{j'+})\}_{j' \in [1, c]}$, then sampling is performed from $\{0, 1\}^n / \mathcal{D}_{S(\cdot, K)}$ without replacement, so no RESET or FCOLL events occur.

If K collides with a record previously generated by queries in Case 4, the situation is similar, and no RESET or FCOLL events occur.

When the i -th query is in Case 4, it means $Y \notin \mathcal{D}_{S(\cdot, K)}$. At this point, $X \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{S(K, \cdot)}$ is sampled. Since queries in Case 2, 3, and 4 all sample and assign the c mappings related to the query $\{(X' \oplus (j-1), K, Y^j)\}_{j \in [1, c]}$, for the X obtained by the i -th sampling where $X \notin \mathcal{D}_{S(K, \cdot)}$, we must have: $\forall j \in [1, c], X \oplus (j-1) \notin \mathcal{D}_{S(K, \cdot)}$. Thus, the i -th query cannot collide with records previously generated by queries in Case 2, 3, or 4, i.e., no RESET or FCOLL events occur.

If K collides with the key $\widetilde{K}^{*+} \oplus (j'-1) \parallel K^{M+}$ in the record $(\tilde{X}^+ \oplus (j'-1), \widetilde{K}^{*+} \oplus (j'-1) \parallel K^{M+}, Y^{j'+})$ previously generated by queries in Case 1. According to the conditions for Case 4, we have $Y \neq Y^{j'+}$. According to the simulation sampling rule, after sampling $X \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{S(K, \cdot)}$, we sequentially check for $j = 2, \dots, c-1$ whether there exists Y^j such that $(X \oplus (j-1), K, Y^j) \in \mathcal{D}_S$. If not, we randomly sample and assign $Y^j \xleftarrow{\$} \{0, 1\}^n / \mathcal{D}_{S(\cdot, K)}$. Thus, no RESET occurs.

Therefore, when the i -th query is in Case 3 or 4, no FALSE event occurs. Thus, we have

$$\begin{aligned}
 \Pr[\text{false}(i)] &\leq \max \left\{ (i-1)/2^{bn-(\log_2 c+1)}, c(i-1)/2^{bn}, 0, 0 \right\}, \\
 \Pr[\text{FALSE}(q)] &\leq \sum_{i=1}^q \max \left\{ (i-1)/2^{bn-(\log_2 c+1)}, c(i-1)/2^{bn}, 0, 0 \right\} \\
 &\leq \sum_{i=1}^q (i-1)/2^{bn-(\log_2 c+1)} \\
 &\leq q(q-1)/2^{bn-(\log_2 c+1)}. \\
 \Pr[\text{BAD}(q)] &\leq \Pr[\text{CONNECT}(q)] + \Pr[\text{MCOLL}(q)] + \Pr[\text{FALSE}(q)] \\
 &\leq \frac{cq(q-1)}{2^{bn+1}} + \frac{q(q-1)}{2^{bn+1}} + \frac{q(q-1)}{2^{bn-(\log_2 c+1)}} \\
 &\leq \frac{(2^{\log_2 c+2} + c + 1)q(q-1)}{2^{bn+1}}.
 \end{aligned}$$

3.3 Security Supplement

3.3.1 Message Length Extension Attack

The message length extension attack does not query the full hash algorithm, but constructs an extended message $M' = M \| M^*$. Using the digest of message $M = M_1 \| \dots \| M_l$ as the intermediate chaining value, the attacker continues processing M^* to obtain the corresponding digest. Since uHash uses CF'_{bn} instead of CF_{bn} to process the last message chunk, the digest of message M is $CF'_{bn}(u_l, M_l)$, which will be the intermediate chaining value used in length extension attacks. However, the true intermediate chaining value of message M' is $CF_{bn}(u_l, M_l)$. Due to the collision resistance of the compression function, the digest of M' computed using the incorrect intermediate chaining value will be different from the true digest of M' .

Without loss of generality, Figure 4 shows an example of uHash's resistance to length extension attacks with $M^* = M_{l+1} \| M_{l+2}$.

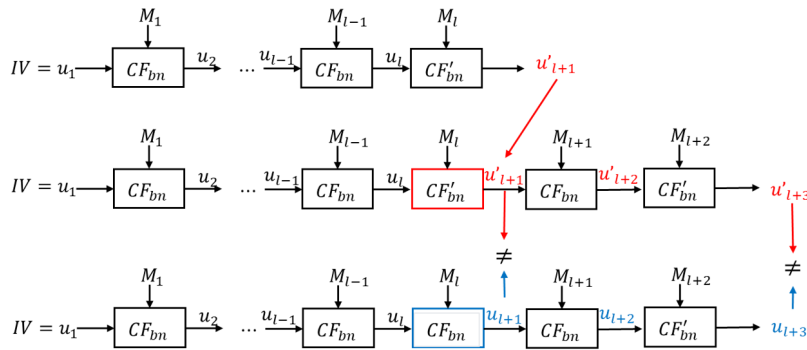


Figure 4: Example of length extension attack

3.3.2 Quantum Security

The quantum security of uHash relies on security conclusions proven via the compressed oracle technique in the quantum random oracle model. Proposed by Zhandry [14], the

compressed oracle technique is regarded as the most reliable security proof method in the quantum random oracle model to date. Its extensions have been applied to prove the indistinguishability of the MD structure and sponge construction, the quantum pseudorandomness of the 4-round Luby–Rackoff structure, and the tight security bound of HMAC; this technique can also be utilized to prove the quantum security of structures such as GFN.

As an application example of the compressed oracle technique, Zhandry proved that for random functions h_1, h_2 , their composition function $C^{h_1, h_2}(x_1, x_2) = h_2(h_1(x_1), x_2)$ (with an n -bit output) is indistinguishable from a random oracle. In other words, the distinguishing advantage for an adversary to distinguish a random function H and C^{h_1, h_2} is bounded by $O(q^{2l}/\sqrt{2^n})$. As a generalization of this theorem, Zhandry quantized the classical indistinguishability definition and corresponding proof techniques [15], and demonstrated that the prefix-free encoded Merkle–Dangård construction satisfies quantum indistinguishability. For a random function $h: \{0, 1\}^m \rightarrow \{0, 1\}^n$, the optimal complexity for quantum preimage search is $\Omega(2^{n/2})$, and the optimal complexity for quantum collision search is $\Omega(2^{n/3})$.

For uHash, each iteration executes b block ciphers in parallel together with feedback operations to construct a one-way round function, with a differentiated processing scheme adopted for the final iteration. Treating each iteration as an individual random function, the final iteration corresponds to a distinct random function. By reusing the security results for composition functions and Merkle–Dangård constructions under the quantum random oracle model, we conclude that the uHash construction is quantum-indistinguishable from a random function, with the distinguishing advantage bounded by $O(q^{2l}/\sqrt{2^{bn}})$. For quantum preimage attacks and collision attacks, the corresponding optimal attack complexity is bounded by $\Omega(2^{bn/3})$.

4 Security of uBlock-1024

This section provides a preliminary security evaluation of the underlying block cipher uBlock-1024 of uHash against classical cryptanalytic methods.

4.1 Differential Cryptanalysis

For differential cryptanalysis [16], considering super S-boxes and the branch number, it can be proven that 4 rounds of encryption contain at least 32 active S-boxes. Thus, 16 rounds contain at least 128 active S-boxes. Since the maximum differential probability of the S-box is 2^{-2} , the maximum differential probability for 16-round algorithm satisfies

$$DCP_{max}^{16r} \leq 2^{-128 \times 2} = 2^{-256}.$$

The longest effective differential path of uBlock-1024 spans 15 rounds, so there is no effective differential distinguisher for full-round algorithm.

We also performed an actual search for the number of active S-boxes. The results show that 4 rounds of encryption contain at least 34 active S-boxes.

4.2 Linear Cryptanalysis

Similar to differential cryptanalysis, considering super S-boxes and the branch number, it can be proven that 4 rounds of encryption contain at least 32 active linear S-boxes [17]. Thus, 16 rounds contain at least 128 active linear S-boxes. Since the maximum linear probability of the S-box is 2^{-2} , the longest effective linear path of uBlock-1024 spans 15 rounds. There is no effective linear distinguisher for full-round algorithm.

4.3 Impossible Differential Cryptanalysis

We use a differential propagation system [18] to search for impossible differential distinguishers of uBlock-1024. Variables are introduced at the nibble level, i.e., all variables take values in $GF(2^4)$. Choosing input and output differences with Hamming weight 1, the search results show the existence of 5-round impossible differentials. There is no effective impossible differential distinguisher for the full-round algorithm.

As an example, let the active position of the input difference be the 0-th nibble, the active position of the output difference after 5 rounds (excluding the linear layer of the last round) is also the 0-th nibble. Thus, the input and output differences form a 5-round impossible differential.

$$\begin{array}{ccc} 10000000000000000000000000000000 & \xrightarrow{5r} & 10000000000000000000000000000000 \\ 00000000000000000000000000000000 & \times & 00000000000000000000000000000000 \end{array}$$

4.4 Integral Cryptanalysis

Using bit-level division property [7, 19, 20], we perform integral cryptanalysis on uBlock-1024 and construct an automated integral path search model based on MILP. The results show that the longest integral distinguisher of the algorithm covers 8 rounds, and there is no effective integral distinguisher for the full-round algorithm.

There are 64 such 8-round integral distinguishers as shown in Table 10. Specifically, choose one nibble to be constant and keep the remaining 63 nibbles active. After 8 rounds of encryption, the XOR sum of all 2^{252} ciphertexts is zero.

Table 10: The 8-round integral distinguishers

Index	Input	Sum of output
1	caaaaaaaaaaaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaaaaaaaaaaa	00000000000000000000000000000000 00000000000000000000000000000000
2	acaaaaaaaaaaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaaaaaaaaaaa	00000000000000000000000000000000 00000000000000000000000000000000
...		
64	aaaaaaaaaaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaaaaaaaaaac	00000000000000000000000000000000 00000000000000000000000000000000

4.5 Meet-in-the-Middle Attack

We evaluate the security of uBlock-1024 against the meet-in-the-middle(MITM) attack based on δ -set [21, 22]. The results show that there exists 6-round distinguishers, and there is no effective distinguisher for the full-round algorithm. Furthermore, we present a 14-round key recovery attack based on a 6-round distinguisher.

Table 11: Parameters of the 14-round MITM attack

Distinguisher	δ -set	Difference sequence	Variables	Forward	Backward
6 rounds	$X^1[11, 15]$	$Y^7[61]$	211	1 round	7 rounds

In this section, the δ -set is a set of 256 states, where the 256 states are active (taking all possible values) only in 2 nibbles and are non-active (identical) in all other positions. Suppose $X^1[11, 15]$, i.e., the 11-th and 15-th nibbles of the distinguisher input are active. Then the 255 4-bit differences:

$$\{Y_0^7[61] \oplus Y_1^7[61], Y_0^7[61] \oplus Y_2^7[61], \dots, Y_0^7[61] \oplus Y_{255}^7[61]\}$$

output by the 256 states after 6 rounds of encryption and one key addition can be determined by 211 4-bit nibbles:

$$\begin{aligned} &Y^1[11, 15], Y^2[19, 23, 24, 26, 28, 31, 35, 36, 37, 38, 40, 45, 46, 47], \\ &Y^3[0, 2, 3, 4, 6 - 63], Y^4, RK_5[0 - 8, 10 - 18, 20, 22, 23 - 49, 51, 52, 55 - 63], \\ &RK_6[25, 27, 28, 29, 30, 31, 56, 57, 60, 61, 62], \end{aligned}$$

where $Y[t] = X[t] \oplus RK[t]$.

Based on the 6-round distinguisher, a 14-round key-recovery attack is constructed by appending 1 round forward and 7 rounds backward (where RK_7 is included in the distinguisher).

Offline phase: Compute the $2^{4 \times 211} = 2^{844}$ possible values of the 255×4 -bit difference sequence and store them in a Table T .

Online phase:

- (1) Choose plaintext P_0 , guess corresponding nibbles of subkey RK^0 , and partially encrypt P_0 to obtain the distinguisher input $X^1[11, 15]$.
- (2) Construct a δ -set at the distinguisher input: let $X^1[11, 15]$ traverse all 256 possible values (specifically, $X^1[11, 15]$ is XORed with 255 non-zero differences), while other positions remain unknown constants.
- (3) Guess corresponding nibbles of RK^0 , and partially decrypt the δ -set to obtain the active differences in the plaintext. XOR these with P_0 to obtain the set of plaintexts that satisfy the distinguisher properties.
- (4) Query the encryption oracle to obtain the ciphertext corresponding to each plaintext.
- (5) Guess the subkeys $RK_8^*[61], RK_9^*[0, 3, 4, 6, 8, 10, 50, 52, 53, 55, 60], RK_{10}^*[0 - 3, 5 - 9, 11 - 34, 36 - 54, 57 - 59, 61 - 63], RK_{11}^*, RK_{12}^*, RK_{13}^*$, where $RK_i^* = M^{-1}(RK_i)$. Decrypt the ciphertext to obtain $Y^7[61]$.
- (6) Check whether the difference sequence of $Y^7[61]$ corresponding to sequence exists in Table T .

Considering equivalent keys of linear key schedule, 227 nibbles need to be guessed during the partial encryption and decryption. After filtering via table lookups, the remaining key space is $2^{(227+211-255) \times 4} = 2^{732}$. Meanwhile, the remaining keys can be further matched with the guessed values of RK_5 and RK_6 from the offline precomputation phase to reduce the remaining key space [23]. The data complexity of the attack is 2^{120} chosen plaintexts, and the computational complexity is approximately $2^{227 \times 4} \times 2^8 = 2^{916}$ partial encryptions/decryptions.

4.6 Boomerang Attack

To evaluate the security of uBlock-1024 against boomerang attack [24], we adopt the sandwich framework $E = E_1 \circ E_m \circ E_0$. First, we construct forward and backward differential active S-box search model for $E_m \circ E_0$ and $E_1 \circ E_m$, respectively. The global objective function is to minimize the total number of active S-boxes in E_0 , E_m , and E_1 , where E_m counts the common active S-boxes in E_0 and E_1 . Second, we instantiate the differential paths for E_0 and E_1 using a bit-level MILP model to determine the corresponding

Table 12: The 14-round MITM attack

	Propagation	Master key involved
Plaintext X^0	00000000 00000000 11111111 11111111 00000000 00000000 01111111 11101111	
$RK_0 S M$		$K_0 K_1$
Distinguisher Input X^1	00000000 00010001 00000000 00000000 00000000 00000000 00000000 00000000	
$RK_1 S M$		$K_2 K_3$
X^2	00000000 00000000 00010001 10101001 00011110 10000111 00000000 00000000	
$RK_2 S M$		$K_4 K_5$
X^3	10111011 11111111 11111111 11111111 11111111 11111111 11111111 11111111	
$RK_3 S M$		$K_6 K_7$
X^4	11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111	
$RK_4 S M$		$K_2 K_5$
X^5	11111111 10111111 11101011 11111111 11111111 11111111 11011001 11111111	
$RK_5 S M$		$K_1 K_0$
X^6	00000000 00000000 00000000 01011111 00000000 00000000 00000000 11001110	
$RK_6 S M (RK_7)$		$K_3 K_6, K_7 K_4$
Distinguisher Output $Y^7 (X^7 \oplus RK_7)$	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000100	
$S RK_8^* M$		$K_1 K_6$
X^8	10011010 10100000 00000000 00000000 00000000 00000000 00101101 00001000	
$S RK_9^* M$		$K_5 K_2$
X^9	11110111 11011111 11111111 11111111 11101111 11111111 11111110 01110111	
$S RK_{10}^* M$		$K_0 K_7$
X^{10}	11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111	
$S RK_{11}^* M$		$K_4 K_3$
X^{11}	11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111	
$S RK_{12}^* M$		$K_5 K_7$
X^{12}	11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111	
$S RK_{13}^* M$		$K_6 K_1$
X^{13}	11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111	

differential propagation probabilities p and q . Finally, based on the instantiated results, we experimentally verify the boomerang switching probability r of the E_m part. The search results show that there exists 7-round boomerang distinguisher with probability $2^{-225.336}$.

$$\Delta_i \xrightarrow{3r} \Delta_o \xrightarrow{2r} \nabla_i \xrightarrow{2r} \nabla_o.$$

$$\begin{aligned} \Delta_i &= 0x66606000060666660660060606606660 \\ &\quad 0x60060600666006600060600066000600, \\ \Delta_o &= 0x00010000000000000000000000000000 \\ &\quad 0x0000000000000000000000000100000000, \\ \nabla_i &= 0x06000600000000000000000000000000 \\ &\quad 0x00000000000000000000000000000000, \\ \nabla_o &= 0x000000000000000000000000010000010000 \\ &\quad 0x00100001000110000000000000000000, \end{aligned}$$

where $p = 2^{-96}$, $q = 2^{-16}$, and $r = 2^{-11.336}$. The sample size traversed in the experiment is 2^{23} .

4.7 Differential-linear Attack

For the differential-linear attack [25], a method similar to the boomerang attack is adopted. First, the algorithm is divided into $E = E_1 \circ E_m \circ E_0$. We construct forward differential active S-box search model for $E_m \circ E_0$ and backward linear active S-box search model for $E_1 \circ E_m$, with the total number of active S-boxes in E_0 , E_m , and E_1 as the global objective function to be minimized. Here, the E_m part counts the common active S-boxes in E_0 and E_1 . Second, we instantiate the differential paths of E_0 and the linear mask paths of E_1 using a bit-level MILP model to determine the corresponding differential propagation probability p and linear correlation q . Finally, based on the instantiated results, we experimentally verify the differential-linear correlation r of the E_m part. The results show that there exists 7-round differential-linear distinguisher with correlation $2^{-114.516}$.

$$\Delta_i \xrightarrow{1r} \Delta_o \xrightarrow{3r} \Gamma_i \xrightarrow{3r} \Gamma_o.$$

$$\begin{aligned} \Delta_i &= 0x000000000000000000000000000000200 \\ &\quad 0x000000000000000000000000000000002, \\ \Delta_o &= 0xa0000000a0a000000000000000000000 \\ &\quad 0x00000000000000000000000000000aaa0, \\ \Gamma_i &= 0x00000000000100000000000000000000 \\ &\quad 0x000000000000000000000000000000100, \\ \Gamma_o &= 0xd055500d055000de055500505000d000 \\ &\quad 0x000500d0505000000d0050d08000d5e, \end{aligned}$$

where $p = 2^{-4}$, $q = 2^{-51}$, and $r = 2^{-8.516}$. The sample size traversed in the experiment is 2^{20} .

5 Performance of uHash

The design of uHash makes full use of register resources and combines a multi-branch parallel block cipher structure, enabling parallel processing on two branches sharing the

same register without requiring multi-threading. The main operations of the underlying block cipher only include 4-bit S-boxes, nibble-wise permutations, and logical operations such as XOR, making it very suitable for implementation using the SIMD instruction sets supported by modern CPUs.

5.1 Software Implementation

We have tested the implementation speed of the uHash algorithm on software platforms. The test environment is: AMD Ryzen 7 5800X 8-Core Processor @ 3.8GHz, with 16.0 GB of memory. The experiments were conducted on the Windows platform, using the MSYS2/MinGW-w64 compilation environment with the g++ 15.2.0 compiler, and compiled under the C++17 standard.

Test results (Table 13 and Table 14) show that the implementation speed of uHash-512 outperforms SHA3-512 at all message lengths; the advantage is particularly prominent in short-message scenarios.

Table 13: Software implementation performance of uHash (cycle/Byte)

Message length	Speed (cycle/Byte)			
	SHA3-512	uHash-512	uHash-768	uHash-1024
8B	228.000	128.250	133.000	137.750
32B	57.000	33.250	33.250	67.688
64B	29.094	16.625	32.656	52.250
68B	27.382	15.647	30.735	48.618
80B	32.775	13.300	26.125	41.800
96B	27.708	21.375	22.167	46.708
100B	25.460	20.900	20.520	43.320
128B	20.188	16.031	24.047	43.047
200B	16.530	14.630	20.330	37.810
300B	16.213	12.920	17.227	36.607
400B	14.250	12.065	17.765	35.150
576B	14.118	11.875	17.417	36.153
1536B	11.702	10.440	16.402	35.180
4096B	10.975	9.927	15.976	34.846
>4096B (1MiB)	10.910	9.859	15.801	34.816

Note: The performance of SHA3-512 was tested under the same environment, linked against OpenSSL's `libcrypto` library.

5.2 Hardware Implementation

To evaluate the real-time processing capability of hash algorithms for continuous raw data streams, this section compares the hardware performance of uHash-512 and SHA3-512 HT (HT stands for the high-throughput implementation where each cycle executes two sequential rounds [26]). A unified hardware efficiency metric is defined as $\eta = \text{Bits} / (L \times GE)$, where L denotes the total clock cycles required for one complete hash operation, GE represents the equivalent gate count synthesized under the Nangate 45 nm process, and η quantifies the data processing efficiency per gate count per clock cycle. The

Table 14: Software implementation performance of uHash (Mbps)

Message length	Speed (Mbps)			
	SHA3-512	uHash-512	uHash-768	uHash-1024
8B	133.333	237.037	228.571	220.690
32B	533.333	914.286	914.286	449.120
64B	1044.889	1828.571	930.916	581.818
68B	1110.218	1942.864	989.100	625.283
80B	927.536	2285.714	1163.636	727.273
96B	1097.156	1422.222	1371.408	650.852
100B	1194.030	1454.545	1481.481	701.754
128B	1505.845	1896.326	1264.191	706.205
200B	1839.081	2077.922	1495.327	804.020
300B	1875.038	2352.941	1764.672	830.442
400B	2133.333	2519.685	1711.230	864.865
576B	2153.280	2560.000	1745.421	840.871
1536B	2597.847	2911.877	1853.433	864.127
4096B	2769.932	3062.355	1902.854	872.410
>4096B (1MiB)	2786.434	3083.477	1923.929	873.162

throughput evaluated at 1 GHz is adopted as an auxiliary performance indicator. This metric enables a comprehensive assessment of hardware area efficiency, avoiding one-sided evaluations that consider only throughput or hardware overhead alone. Specifically, input efficiency η_{in} adopts the per-iteration input message length as the statistical variable, which accurately reflects the hardware capability of handling continuous incoming data and is more suitable for high-speed pipeline transmission and online data verification scenarios. We synthesize our design using the open-source Yosys 0.64 for reproducibility; note that the reported performance could be further optimized with advanced commercial EDA tools like Synopsys Design Compiler.

As the baseline design, SHA3-512 HT supports a 344-bit input block with a latency of 22 clock cycles and a gate count of 47914, yielding an input efficiency of $\eta_{\text{in}} = 3.26 \times 10^{-4}$ and an input throughput of 15.6 Gbps at 1 GHz.

uHash-512 employs an expanded 768-bit input block and reduces the total latency to 16 clock cycles, with a moderately increased gate count of 59087. It is worth noting that the underlying block cipher uBlock-1024 updates the master key every four rounds to generate corresponding round subkeys, and the reported hardware area of uHash-512 corresponds to the fully unrolled implementation of four-round uBlock-1024 encryption. uHash-512 achieves an input efficiency of $\eta_{\text{in}} = 8.12 \times 10^{-4}$, which is 149% higher than that of SHA3-512 HT, and delivers a peak input throughput of 48.0 Gbps at 1 GHz. Overall, uHash-512 achieves a substantial improvement in input processing throughput and hardware resource utilization with acceptable area overhead, demonstrating superior adaptability for high-speed continuous data streaming applications.

5.3 Other Platforms

Designed upon a lightweight block cipher, uHash is adaptable to various software and hardware platforms. On PCs, it can be efficiently implemented using instruction sets such

Table 15: Hardware Implementation Performance of uHash (Nangate 45 nm, 1 GHz)

Algorithm	GE	L	η_{in}	Throughput
SHA3-512 HT	47914	22	3.26×10^{-4}	15.6 Gbps
uHash-512	59087	16	8.12×10^{-4}	48.0 Gbps

L : Clock cycles for one complete hash computation; GE: Equivalent gate count under Nangate 45 nm process; $\eta_{in} = \text{Input Bits}/(L \times GE)$: Hardware area efficiency.

as SSE and AVX2; especially if AVX-512 is available, software performance of uHash can be further optimized; on mainstream ARM platforms, it can be optimized using the NEON instruction set. The use of 4-bit low-latency S-boxes satisfies the demands of low-latency applications. The main operations are limited to 4-bit S-boxes, nibble-wise permutations, and XOR operations, making it highly suitable for 8-bit microcontrollers. For hardware, the parallel structure, modular design, simple S-boxes, and linear transformations make uHash straightforward and efficient to implement, supporting both high-speed and lightweight implementations.

We also tested the performance of uHash on Hygon. The testing environment consisted of a Hygon C86-4G processor running Kylin Linux Advanced Server V10, utilizing g++ 7.3.0 and OpenSSL 1.1.1f. The benchmark was pinned to Core 32 on NUMA Node 2. The results demonstrate that uHash-512 still maintains speed advantage over SHA3-512.

Table 16: Software implementation performance of uHash on Hygon(cycle/Byte)

Message length	Speed (cycle/Byte)			
	SHA3-512	uHash-512	uHash-768	uHash-1024
32B	50.625	44.877	69.531	123.483
128B	20.766	21.156	42.991	72.163
512B	17.057	15.131	30.278	60.909
1024B	15.407	13.644	28.391	58.475
4096B	14.132	13.323	26.624	56.023
8192B	14.052	13.443	26.356	57.082
16384B	14.063	13.167	26.141	56.220
65536B	14.003	13.177	26.028	55.927

6 Features of uHash

uHash is designed based on block cipher. Through multi-block parallel structure, it realizes a cryptographic hash algorithm with flexibly controllable high security levels. Algorithms with different parameters are built on the same framework, facilitating easy extension. The compression function adopts a variant of the Counter-bDM structure. Under the ideal cipher model, uHash admits provable security in terms of collision resistance, preimage resistance, and second-preimage resistance, and its output digests are computationally indistinguishable from uniformly random bitstrings of the same length.

The underlying block cipher uBlock-1024 is based on the design philosophy of uBlock-256 with targeted optimizations, offering provable security against differential and linear cryptanalysis, as well as sufficient security margin against other classical cryptanalytic methods. As the winner of the National Cryptographic Algorithm Design Competition,

uBlock has undergone years of extensive public security analysis and evaluation, with its performance further improved through various optimizations. uBlock-1024 inherits most of uBlock's core security properties. By leveraging the established security guarantees of uBlock, it further underpins the security of uHash.

uHash makes full use of register resources and combines a multi-block parallel structure, enabling parallel processing on two branches sharing the same register without multi-threading, thus avoiding resource waste. The overall structure is oriented towards SSE/AVX instruction sets, resulting in fast software implementation speed. In particular, uHash-512 significantly outperforms SHA3-512 in processing short messages. In post-quantum cryptography deployments, zero-knowledge proof systems, and high-security-level authentication protocols, the requirement for 512-bit preimage resistance (equivalent to SHA3-512) rarely comes with multi-megabyte-level data streams. In high-security scenarios, inputs no longer than 1023 bits (0–128 bytes) account for 85%–98% of the total calls to the hash function. Different-scale compression functions in uHash reuse the same underlying block cipher, and the parameter design with a fixed 1024-bit key effectively improves the message processing rate. In addition, the multi-block parallel structure provides leverage for further optimization of the software implementation in multi-threaded environments.

7 Statistical Tests of uHash

the statistical test items and parameters are set according to the cryptographic hash algorithm test specification.

7.1 Statistical Test of uHash-512

Significance level: 0.01

Sample generation method: counter mode

Number of message chunks: 1000000

Number of tests: 1000

7.2 Statistical Test of uHash-768

Significance level: 0.01

Sample generation method: high Hamming weight

Number of message chunks: 1000000

Number of tests: 1000

7.3 Statistical Test of uHash-1024

Significance level: 0.01

Sample generation method: low Hamming weight

Number of message chunks: 1000000

Number of tests: 1000

8 Test Vectors of uHash

The test vectors of uHash are used to verify the correctness of the implementation. All data are represented in hexadecimal. Some examples are given below; complete test vectors can be found in the "Test_Vectors" folder.

Table 17: Statistical test results of uHash-512

Test item	Rate (%)
Single-bit Frequency	98.90
Block Frequency (100)	98.90
Poker (4)	99.00
Overlapping Sub-sequence (5)-1	99.00
Overlapping Sub-sequence (5)-2	98.80
Total Runs	98.90
Run Distribution	99.20
Binary Derivation (7)	99.30
Autocorrelation (2)	98.10
Matrix Rank	99.60
Cumulative Sum(Forward)	99.10
Cumulative Sum(Backward)	99.00
Approximate Entropy (5)	98.60
Linear Complexity (500)	99.10
Universal Statistical (7) (1280)	99.20
Discrete Fourier Transform	98.80

Table 18: Statistical test results of uHash-768

Test item	Rate (%)
Single-bit Frequency	98.40
Block Frequency (100)	98.80
Poker (4)	99.50
Overlapping Sub-sequence (5)-1	99.00
Overlapping Sub-sequence (5)-2	99.10
Total Runs	99.10
Run Distribution	99.00
Binary Derivation (7)	98.80
Autocorrelation (2)	98.30
Matrix Rank	98.30
Cumulative Sum(Forward)	98.30
Cumulative Sum(Backward)	98.10
Approximate Entropy (5)	98.80
Linear Complexity (500)	98.80
Universal Statistical (7) (1280)	98.90
Discrete Fourier Transform	98.80

Table 19: Statistical test results of uHash-1024

Test item	Rate (%)
Single-bit Frequency	99.20
Block Frequency (100)	98.90
Poker (4)	98.70
Overlapping Sub-sequence (5)-1	98.90
Overlapping Sub-sequence (5)-2	99.00
Total Runs	98.40
Run Distribution	99.10
Binary Derivation (7)	99.10
Autocorrelation (2)	99.20
Matrix Rank	99.00
Cumulative Sum(Forward)	99.00
Cumulative Sum(Backward)	99.00
Approximate Entropy (5)	99.10
Linear Complexity (500)	99.00
Universal Statistical (7) (1280)	99.00
Discrete Fourier Transform	99.10

8.1 Test Vectors of uHash-512

- Example 1:
 Msg_Len = 0
 Msg =
 Dst_Len = 512
 Dst =
 54DEDA01CFFC8ECA07114698A114CA828DD3873462F36274DC196CAB000CF4EAFD109
 D7AE9D70BDF59D9F71CCD5AD8E506215D5D8AD1F546ECFFE9FC1866DAAA
- Example 2:
 Msg_Len = 8388608
 Msg_Seed =
 Msg_Exp = FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFF
 Dst_Len = 512
 Dst =
 A0C1EEC982509259816DEE886662D421BEFB4CA88EFC1C15B6CE01D4B4678C586572C
 6747453577D33D03622339DB9B6D855032F2AF9C0A59D1CA162C1A555CF
- Example 3:
 Msg_Len = 8589934592
 Msg_Seed =
 4B41545F325F33334B41545F325F33334B41545F325F33334B41545F325F33334B415
 45F325F33334B41545F325F33334B41545F325F33334B41545F325F3333
 Msg_Exp = AE6416F9B6D4C97A 31B80BC6E737A914
 Dst_Len = 512
 Dst =
 64D067093A4F9F81237F18394A575AB8DD799671A18EB1E7824E9F05931015F751928

D2C450B9F2B6EA323BCA0854237D97BBC1C0BE026454DF24E317EF8951D

- Example 4:
 Msg_Len = 8192
 Msg =
 2D911820A8ED087902CCFEEAD2C91785CEA0289755C29F651DB41AF1D8434774FCA39
 1F2F7F9473A2619DC75FCC817E42B7A591AF2E95E3D43EF7D4EA978EEABF0F2E6997B
 64CE29E776A297B5A8EDBC10C7804C2EBF0F76DBBD78EBEFAB289629B5A9BBA6B2F0F
 87D59D73462F3076F35B6C24B47A2B5621A7C68274CE1E54F0388BE2FB29051FC9E3B
 7E9E137D4CAB088849D39ACBF9D919F0CC38F95133345E329F79C7E12D1B3ADFB2F62
 A42CF16D2489BCB8351016AB5C2F896397BED83BF9749FB9E7F6AA0B13B16D0A6242D
 9AA207B0AA23F0457C8DCAC96014727ABD5193AA95E65CBF2021D8BCE16DB4ECC701E
 B3F734A4AAD55F6F8D6BD47B9E426443C557AF8EF48ABE5E9995FBA06E0B13F116F4C
 BB11B1E82EFF5B346D85DF49A89D515D4BFE2C901E41CFDDF463AA5C5E2354E934E4
 47FBBC28FFB31FA2C3F8A8056E4DF8E51FF22C88EA9F78ED0633A2EDD5095AF33FC68
 5916DB679C9A1DF1D2EAC1A5F8200093F1CCCE807B1D46BEECE6CECEF7BF74F39EF21
 A332D4EB75B96953387CA485D3C428C392B86928D565886525F9AF84A0283F0598181
 3C7300DA103F39B5F9C3DDFCFA401AD90C5759FDCB56EA2597D36114A2A3DE0BB44FD
 5472C712EFA1188C6D60C4FCCD3BECCF1887DAD096BAC54689DED755ED453A4C1CEOC
 A981288C2EA0D7EB4E71C53C250DF0C876CFBC45F6AB2821EC8FE0ED5AC5293E644C4
 931FD03E0702FB3478E1FFC39A96D4C06035D30B72F3A5FE6233945A3A515FB55D419
 DF2EFBA5E358C65E33FE54C158B6215363BBD338A9EC3307821865FCE5BFCFBF4B4F1
 A8C5FBD446469A1A65CF45973BDFE3C4A26F5EDFECDF226E50F2A11A664CA45B8E20B
 A412ECE8A5FEE323B69F5835EEAA3BB95DDE9F11291E7473F46AE99A9F0413F855D05
 EBD57E597D45DC50B792E7881E51D84C77EDC8016F6864971A0EA72876761A8E523F1
 AFB1254DD2A399BE12DC6B0FA38174D292990114924468B6D86845A20701EAC464AC9
 887702C0B5B130288D08F93AF671E66DAAD02D64B60C61CE47066BC7AFEDA951B3170
 798F357E4E28D3CC123B29621AB500FE112C05D2BC66A98AB42FF129A19A6D39413D3
 F58672D3EB6C0ADE923F2A2DB9E8D847B53929D32397D380AC1A4835CB11E632C4CA
 5B73ED02885717116B8EB2010CED404F9CA517D41D7F25AB2EC0F3FF4D81EEF0C9060
 2E284AC86EE466A2E18525BEFDE229283599A2161308A685C0E35A26C480BAA36C872
 50077E93D75B4B6930D3C5A9CDFB8E08652BFE4DB82CC50D6D37AE3B83BFB63C3A4D8
 AC4CF5AF7BE1DA11EABCEB3C20B16517650CB28E5CE5E23AD22DF395A7FEE3FCCC154
 0FED322954AECEBAC6C3F0F94099A532847B9E4F888A37716FDCE660CC13122AC974D
 314E4A924BDB550FFC029E36FAC6D91E7A807DBC461377D
 Dst_Len = 512
 Dst =
 605EE98AFED48D4C1E6DC4E3A3B590A7C23140376631FFEBDAB97AC01998877755EF2
 B7D4A977F1E9028C0D56AA786D4D83C7595852811516B07252EE2CBF84BD2C450B9F2
 B6EA323BCA0854237D97BBC1C0BE026454DF24E317EF8951D

8.2 Test Vectors of uHash-768

- Example 1:
 Msg_Len = 0
 Msg =
 Dst_Len = 768
 Dst =
 1702A336338922CFE6CEE74B1799782C4586776D803E40A3525BE337CC1DF08158C96
 34D5CAA19843544EDOC7DFD81F398478F2884871A6DCC6FF063439EE133456EB3A6CB
 23E8E5022B5A888B7B19EEDDC59446A102773FED92795E2B6DFF87
- Example 2:
 Msg_Len = 8388608

```

Msg_Seed =
Msg_Exp = FFFFFFFFFFFFFFFF ..... FFFFFFFFFFFFFFFF
Dst_Len = 768
Dst =
D1FE556E1805B4E1A80A75759EE9FAAA7D9A93557B7C6CODE81206D40547F218E26C8
4AC4AC5F9002790874659211DF10F4D9AE74FBCAB0EE2E91B39F7C5959AD16A62A4AC
7C6E7373D059D0E5F80B9A6C5E4CC83C24C274B17574C2ECD8E6F3

```

- Example 3:

```

Msg_Len = 8589934592
Msg_Seed =
4B41545F325F33334B41545F325F33334B41545F325F33334B41545F325F33334B415
45F325F33334B41545F325F33334B41545F325F33334B41545F325F3333
Msg_Exp = AE6416F9B6D4C97A ..... 31B80BC6E737A914
Dst_Len = 768
Dst =
752ED27839D88DFA9F4EDE66C5C31724AA4F70DDBAA6599A890938930A7F6B5CC6CFB
85DACA7F5CC00AE2C460E6854AB7B9477D80BDD9A3614530698BD1B71DD05716673BA
785484800534ED6A45B71EF21F5CC55746845C55552439FC041AA4

```

- Example 4:

```

Msg_Len = 8192
Msg =
2D911820A8ED087902CCFEEAD2C91785CEA0289755C29F651DB41AF1D8434774FCA39
1F2F7F9473A2619DC75FCC817E42B7A591AF2E95E3D43EF7D4EA978EEABF0F2E6997B
64CE29E776A297B5A8EDBC10C7804C2EBF0F76DBBD78EBEFAB289629B5A9BBA6B2F0F
87D59D73462F3076F35B6C24B47A2B5621A7C68274CE1E54F0388BE2FB29051FC9E3B
7E9E137D4CAB088849D39ACBF9D919F0CC38F95133345E329F79C7E12D1B3ADFB2F62
A42CF16D2489BCB8351016AB5C2F896397BED83BF9749FB9E7F6AA0B13B16D0A6242D
9AA207B0AA23F0457C8DCAC96014727ABD5193AA95E65CBF2021D8BCE16DB4ECC701E
B3F734A4AAD55F6F8D6BD47B9E426443C557AF8EF48ABE5E9995FBA06E0B13F116F4C
BB11B1E82EFFF5B346D85DF49A89D515D4BFE2C901E41CFDDF463AA5C5E2354E934E4
47FBBC28FFB31FA2C3F8A8056E4DF8E51FF22C88EA9F78ED0633A2EDD5095AF33FC68
5916DB679C9A1DF1D2EAC1A5F8200093F1CCCE807B1D46BEECE6CECEF7BF74F39EF21
A332D4EB75B96953387CA485D3C428C392B86928D565886525F9AF84A0283F0598181
3C7300DA103F39B5F9C3DDFCFA401AD90C5759FDCB56EA2597D36114A2A3DE0BB44FD
5472C712EFA1188C6D60C4FCCD3BECCF1887DAD096BAC54689DED755ED453A4C1CE0C
A981288C2EA0D7EB4E71C53C250DF0C876CFBC45F6AB2821EC8FE0ED5AC5293E644C4
931FD03E0702FB3478E1FFC39A96D4C06035D30B72F3A5FE6233945A3A515FB55D419
DF2EFBA5E358C65E33FE54C158B6215363BBD338A9EC3307821865FCE5BFCFBF4B4F1
A8C5FBD446469A1A65CF45973BDFE3C4A26F5EDFECDF226E50F2A11A664CA45B8E20B
A412ECE8A5FEE323B69F5835EEAA3BB95DDE9F11291E7473F46AE99A9F0413F855D05
EBD57E597D45DC50B792E7881E51D84C77EDC8016F6864971A0EA72876761A8E523F1
AFB1254DD2A399BE12DC6B0FA38174D292990114924468B6D86845A20701EAC464AC9
887702C0B5B130288D08F93AF671E66DAAD02D64B60C61CE47066BC7AFEDA951B3170
798F357E4E28D3CC123B29621AB500FE112C05D2BC66A98AB42FF129A19A6D39413D3
F58672D3EB6C0ADE923F2A2DB9E8D847B53929D32397D380AC1A4835CB11E632C4CA
5B73ED02885717116B8EB2010CED404F9CA517D41D7F25AB2EC0F3FF4D81EEF0C9060
2E284AC86EE466A2E18525BEFDE229283599A2161308A685C0E35A26C480BAA36C872
50077E93D75B4B6930D3C5A9CDFB8E08652BFE4DB82CC50D6D37AE3B83BFB63C3A4D8
AC4CF5AF7BE1DA11EABCEB3C20B16517650CB28E5CE5E23AD22DF395A7FEE3FCCC154
0FED322954AECEBAC6C3F0F94099A532847B9E4F888A37716FDCE660CC13122AC974D
314E4A924BDB550FFC029E36FAC6D91E7A807DBC461377D

```

```

Dst_Len = 768
Dst =
87BFC49977B57B5480BEA7A1CAC0A257DF0184895EDD357935F0838D666633DCAC9F0
0BFC27F32DC19FB80A4B1EA4841A98E4E6D4DF1474A57611E40057DE794BFC66C2113
BF68830E01728CC020CDB1ABD411EBA4BC6E5FF141C79F2557F493

```

8.3 Test Vectors of uHash-1024

- Example 1:
 Msg_Len = 0
 Msg =
 Dst_Len = 1024
 Dst =
 5BAD7ACAC904DB05EAC79A24C20A71F7B08D4F4742826B0A88C8A92B1B11F5326ADF0
 AE60417BA2900CCDBFA50CB38BA49B0C96AB8CEB9180A44E0743C633033DC8E67BE23
 27691C801384FBB4FE2831681B8A5A9D8967CB34D6C398FA9AC6808872BFDCD3D4686
 048AC9215D8AD1210F04403EDC01DC30B90A1B41B96DE06A7
- Example 2:
 Msg_Len = 8388608
 Msg_Seed =
 Msg_Exp = FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF
 Dst_Len = 1024
 Dst =
 637350140F03DAE5EBF878150445AA67998D26D93236D4EC95638B06ECA4DF9652B13
 056410C7BE1DA60C88729FABF66250A4A2C0E90993DAA8D290BEEC2B98FD892512D25
 A361B71661E7327F100207930F49078C11C41A03FE0169988D918D6514D2DF8CFE35B
 D2E0DC01FBE7325A5D501EC2328DBE8F8CB485A71D8E700A2
- Example 3:
 Msg_Len = 8589934592
 Msg_Seed =
 4B41545F325F33334B41545F325F33334B41545F325F33334B41545F325F33334B415
 45F325F33334B41545F325F33334B41545F325F33334B41545F325F3333
 Msg_Exp = AE6416F9B6D4C97A 31B80BC6E737A914
 Dst_Len = 1024
 Dst =
 94F91A800D510D7F984B1738E96B2FFADB55EC49434B14ECC4557FA1946529F111055
 D8AF87132FF3BDC0EFA9332346A3ECC73281C633CF60EFE3F87187F36105F424E4FDB
 D731EDB2272AC06823F021593D79F13F9F8CBD419EC1DA49887AF20A42BEB7EE534E1
 6A3B3DDCB392CDA4C2EAFCB89B31B70D843B0AE017A0B8722
- Example 4:
 Msg_Len = 8192
 Msg =
 2D911820A8ED087902CCFEEAD2C91785CEA0289755C29F651DB41AF1D8434774FCA39
 1F2F7F9473A2619DC75FCC817E42B7A591AF2E95E3D43EF7D4EA978EEABF0F2E6997B
 64CE29E776A297B5A8EDBC10C7804C2EBF0F76DBBD78EBEFAB289629B5A9BBA6B2F0F
 87D59D73462F3076F35B6C24B47A2B5621A7C68274CE1E54F0388BE2FB29051FC9E3B
 7E9E137D4CAB088849D39ACBF9D919F0CC38F95133345E329F79C7E12D1B3ADFB2F62
 A42CF16D2489BCB8351016AB5C2F896397BED83BF9749FB9E7F6AA0B13B16D0A6242D
 9AA207B0AA23F0457C8DCAC96014727ABD5193AA95E65CBF2021D8BCE16DB4ECC701E
 B3F734A4AAD55F6F8D6BD47B9E426443C557AF8EF48ABE5E9995FBA06E0B13F116F4C
 BB11B1E82EFFF5B346D85DF49A89D515D4BFE2C901E41CFDDF463AA5C5E2354E934E4

```

47FBBC28FFB31FA2C3F8A8056E4DF8E51FF22C88EA9F78ED0633A2EDD5095AF33FC68
5916DB679C9A1DF1D2EAC1A5F8200093F1CCCE807B1D46BEECE6CECEF7BF74F39EF21
A332D4EB75B96953387CA485D3C428C392B86928D565886525F9AF84A0283F0598181
3C7300DA103F39B5F9C3DDFCFA401AD90C5759FDCB56EA2597D36114A2A3DE0BB44FD
5472C712EFA1188C6D60C4FCCD3BECCF1887DAD096BAC54689DED755ED453A4C1CE0C
A981288C2EA0D7EB4E71C53C250DF0C876CFBC45F6AB2821EC8FE0ED5AC5293E644C4
931FD03E0702FB3478E1FFC39A96D4C06035D30B72F3A5FE6233945A3A515FB55D419
DF2EFBA5E358C65E33FE54C158B6215363BBD338A9EC3307821865FCE5BFCFBF4B4F1
A8C5FBD446469A1A65CF45973BD FE3C4A26F5EDFECDF226E50F2A11A664CA45B8E20B
A412ECE8A5FEE323B69F5835EEAA3BB95DDE9F11291E7473F46AE99A9F0413F855D05
EBD57E597D45DC50B792E7881E51D84C77EDC8016F6864971A0EA72876761A8E523F1
AFB1254DD2A399BE12DC6B0FA38174D292990114924468B6D86845A20701EAC464AC9
887702C0B5B130288D08F93AF671E66DAAD02D64B60C61CE47066BC7AFEDA951B3170
798F357E4E28D3CC123B29621AB500FE112C05D2BC66A98AB42FF129A19A6D39413D3
F58672D3EB6C0ADE923F2A2DB9E8D847B53929D32397D380AC1A4835CB11E632C4CA
5B73ED02885717116B8EB2010CED404F9CA517D41D7F25AB2EC0F3FF4D81EEF0C9060
2E284AC86EE466A2E18525BEFDE229283599A2161308A685C0E35A26C480BAA36C872
50077E93D75B4B6930D3C5A9CDFB8E08652BFE4DB82CC50D6D37AE3B83BFB63C3A4D8
AC4CF5AF7BE1DA11EABCEB3C20B16517650CB28E5CE5E23AD22DF395A7FEE3FCCC154
0FED322954AECEBAC6C3F0F94099A532847B9E4F888A37716FDCE660CC13122AC974D
314E4A924BDB550FFC029E36FAC6D91E7A807DBC461377D
Dst_Len = 1024
Dst =
A196FDE65EA93D0526E899749EA5C96B8208CD8881C54EC0AF1FFDDCBEDE1AE9C3757
8FF70F8206918FFBD3C52DC6A0E0815CB1B28B189BCDDF7F474EBB99FF6A8030C2CF6
108793D48A3119396AAD2813B4D53A4AA051552F7B344CF1A9FAA15F4B051FFA1EB75
0C91A851D92EA74B3E0780917ED86EEEA7A828C1B771CA45

```

References

- [1] Wu W L, Zhang L, Zheng Y F, Li L C. The block cipher uBlock. *Journal of Cryptologic Research*, 2019, 6(6): 690-703.
- [2] Abed F, Forler C, List E, Lucks S, Wenzel J. Counter-bDM: A provably secure family of multi-block-length compression functions. In: *AFRICACRYPT 2014*, LNCS 8469, pp. 440–458. Springer, Heidelberg, 2014.
- [3] Li Y J, Xu X L. Proof of impossible differential distinguishers for a class of SP structures. *Application Research of Computers*, 2021, 38(5): 1529–1532.
- [4] Zhang L, Zhang Y, Wu W, Mao Y, Zheng Y. Explicit upper bound of impossible differentials for AES-like ciphers: applications to uBlock and Midori. *The Computer Journal*, 2023: 1–12.
- [5] Tian W, Hu B. Integral cryptanalysis on two block ciphers Pyjamask and uBlock. *IET Information Security*, 2020, 14(5): 572–579.
- [6] Li X D, Wu W L, Zhang L. Efficient search for optimal vector permutation of uBlock-like structures. *Journal of Computer Research and Development*, 2022, 10: 2275–2285.
- [7] Mao Y, Wu W, Wang B, Zhang L. Improved division property for ciphers with complex linear layers. In: *ACISP 2022*, pp. 106–124, 2022.

- [8] Wang C, Wang J, Li M, et al. New strategy for evaluating differential clustering effect of ublock. *Lecture Notes in Computer Science, Volume 14896, Information Security and Privacy - 29th Australasian Conference, ACISP 2024*: 205–224.
- [9] Zhang Y, Wu W L, Zhang L, et al. Improved modeling methods for automated differential analysis of block ciphers: applications to ublock-256 and PRESENT. *Chinese Journal of Electronics*, 2026, 35(3): 1–16.
- [10] Wang Q, Lu J. Fault analysis of the ARIA and uBlock block ciphers. *IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI)*, IEEE, 2021: 1–6.
- [11] Gong Z R, Guo H, Chen C, et al. FBS-uBlock: Flexible Bit Slicing Optimization Method of uBlock Algorithm. *Journal of Software*, 2025, 36(10): 4827–4845.
- [12] Wu R C, Zhang L. Research on construction of low-latency S-boxes and bidirectional low-latency properties. *Acta Electronica Sinica*, 2024, 52(11): 3769–3779.
- [13] Guo Z, Wu W, Gao S. Constructing lightweight optimal diffusion primitives with Feistel structure. In: *SAC 2015, LNCS 9566*, pp. 352–372. Springer, Heidelberg, 2016.
- [14] Zhandry, M.. How to Record Quantum Queries, and Applications to Quantum Indifferentiability. In *Advances in Cryptology – CRYPTO 2019. Lecture Notes in Computer Science*, vol 11693, pp.239–268.
- [15] Coron, JS., Dodis, Y., Malinaud, C., Puniya, P.. Merkle-Damgård Revisited: How to Construct a Hash Function. In *Advances in Cryptology – CRYPTO 2005. Lecture Notes in Computer Science*, vol 3621, pp.430–448.
- [16] Biham E, Shamir A. *Differential Cryptanalysis of the Data Encryption Standard*. Springer, Berlin, 1993.
- [17] Matsui M. Linear cryptanalysis method for DES cipher. In: *EUROCRYPT 1993, LNCS 765*, pp. 386–397. Springer, Heidelberg, 1994.
- [18] Wu S, Wang M. Automatic search of truncated impossible differentials for word-oriented block ciphers. 2012.
- [19] Xiang Z, Zhang W, Bao Z, et al. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In: *ASIACRYPT 2016, LNCS 10031*, pp. 648–678. Springer, Heidelberg, 2016.
- [20] Todo Y, Morii M. Bit-based division property and application to Simon family. In: *FSE 2016, Revised Selected Papers*, 2016.
- [21] Demirci, H., Taskin, I., C¸oban, M., et al: Improved meet-in-the-middle attacks on AES. In: *INDOCRYPT 2009. LNCS*, vol. 5922, pp. 144–156. Springer, Heidelberg (2009).
- [22] Lin, L., Wu, W., Zheng, Y.: Improved meet-in-the-middle distinguisher on feistel schemes. In: *SAC 2015. LNCS*, vol. 9566, pp. 122–142. Springer, Heidelberg (2015).
- [23] Li Lin, Wenling Wu, and Yafei Zheng. Automatic Search for Key-Bridging Technique: Applications to LBlock and TWINE. *Fast Software Encryption*, 2016. 9783:247–267.
- [24] C. Cid, T. Huang, T. Peyrin, Y. Sasaki, L. Song. Boomerang Connectivity Table: A New Cryptanalysis Tool. In *Advances in Cryptology–EUROCRYPT 2018*, LNCS 10821, pp.683–714, Springer, 2018.

-
- [25] A. Bar-On, O. Dunkelman, N. Keller, A. Weizman. DLCT: A New Tool for Differential-linear Cryptanalysis. In *Advances in Cryptology-EUROCRYPT 2019*, LNCS 11477, pp.313–342, Springer, 2019.
 - [26] H. Hsing. SHA-3 Core (Keccak). OpenCores / GitHub freecores organization, available at <https://github.com/freecores/sha3>, Online; accessed 27-June-2026, 2013.